Dissertation Defense

Optimizing & *Fortifying* AI Software through the Lens of Artifact Synthesis

Jieke Shi
Final-year PhD Candidate at SMU
AI & Software

jiekeshi@smu.edu.sg
https://jiekeshi.tech

SMU
SINGAPORE MANAGEMENT UNIVERSITY

Committee Members:

David LO (Supervisor/Chair)
Lingxiao JIANG
Xiaofei XIE
Zhenchang XING

Self-Introduction

SHI Jieke (史杰克)

https://jiekeshi.tech

Self-Introduction

SHI Jieke (史杰克)

https://jiekeshi.tech

Research Scientist (2026.04, expected)
Ph.D. Candidate (2022.08 - now)
Research Engineer (2021.06 - now)
Singapore Management University

Research Assistant (2020.07 - 2021.04)
Chinese Academy of Sciences

Undergraduate Study (2016.09-2020.06)
Yangzhou University

Research Achievements

I publish papers ☺

30 Papers Published:

- 8 First/Co-first Authored
- 3 as Corresponding Author

Research Achievements

I publish award-winning and highly-cited papers ☺

30 Papers Published:
- 8 First/Co-first Authored
- 3 as Corresponding Author

ACM SIGSOFT Distinguished Paper Awards
(ASE'22 nomination, ASE'25 winner, ICSE'26 winner)

IEEE Computer Society TCSE Distinguished Paper Award (SANER'26)

IEEE Computer Society Best Paper Award (TSE'24)
Honorable Mention Award (ACSAC'22)

Most Cited Paper (among all accepted papers in ICSE'24 SEIS track)

Research Achievements

I get small grants and contribute to OSS community ☺

I design methods for optimizing and validating software.

30 Papers Published:
- 8 First/Co-first Authored
- 3 as Corresponding Author

Google Cloud Research Credits Grant ($2500)

220+ bugs in open-source software/compilers, with 30+ PRs to fix them

Optimizing Software

Model Compression/Acceleration

-- Making AI models smaller and faster

- Code Model Compression (ASE 2022) 🏆
- Code Model Greening (ICSE 2024) 🏆
- Vocabulary Minimization (SANER 2022)

LLM Ecosystem & Efficiency Survey

-- Understanding the AI software landscape

- Green LLM Survey (TOSEM 2025)
- Code LLM Ecosystem (TOSEM 2025)
- LLM-as-a-Judge Survey (TOSEM 2026)

Developer Experience & Productivity

-- How developers use/write software

- AI Assistant Perception (ASE 2025)
- Community Code Reasoning (ICSE 2026) 🏆
- AI Repo Issue Study (MSR 2023)
- Technical QA Summarization (ASE 2022)

Validating Software

Autonomous System Safety

-- Shielding and testing AI-driven agents

- Safety Violation Detection (TOSEM 2025)
- Runtime Shield Synthesis (TOSEM 2025)
- RL Curiosity Testing (ICSE 2024)
- Adversarial RL Policies (ACSAC 2022) 🏆

Adversarial Robustness & Privacy

-- Exposing attacks and leakage in code models

- Natural Code Attack (ICSE 2022)
- Stealthy Backdoor Attack (TSE 2024) 🏆
- RL Backdoor Injection (S&P 2024)
- Code Membership Leakage (TSE 2024)

Trusted Execution & System Security

-- Securing TEE system or Web system

- TEE Misuse Study (SANER 2026) 🏆
- TEE Input Validation (FORGE 2026)
- Automated Pen Testing (ICSE 2026)

Software Quality & Testing

-- Improving testing and bug localization

- Neuron Coverage Revisit (SANER 2022)
- Speech Test Prioritization (TOSEM 2024)
- Issue-Commit Linking (ICSE 2026)
- Bug Localization (SANER 2022, ASE 2021)

Dissertation Defense

Optimizing & *Fortifying* AI Software through the Lens of Artifact Synthesis

Jieke Shi
Final-year PhD Candidate at SMU
AI & Software

jiekeshi@smu.edu.sg
https://jiekeshi.tech

SMU
SINGAPORE MANAGEMENT UNIVERSITY

Committee Members:

David LO (Supervisor/Chair)
Lingxiao JIANG
Xiaofei XIE
Zhenchang XING

Artifact Synthesis for AI4SE Software

the automated generation of software artifacts (configurations, programs, shields) that satisfy given specifications

Optimizing

in terms of efficiency

Compressing AI4SE Software via
Hyperparameter Synthesis
(ASE 2022)

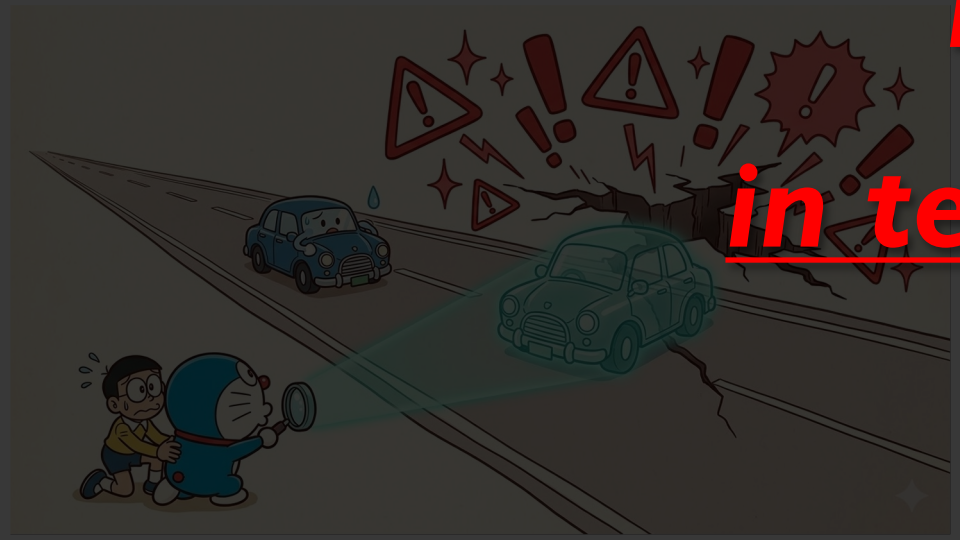
Kernel Synthesis
(FSE 2026)

Greening AI4SE Software via
Configuration Synthesis
(ICSE 2024)

Artifact Synthesis for AI4Control Software

the automated generation of software artifacts (configurations, programs, shields) that satisfy given specifications

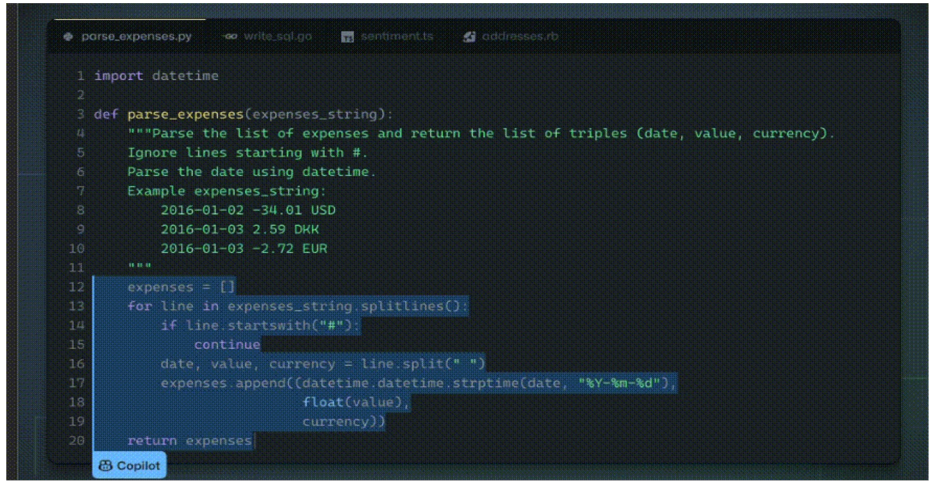
Fortifying in terms of safety

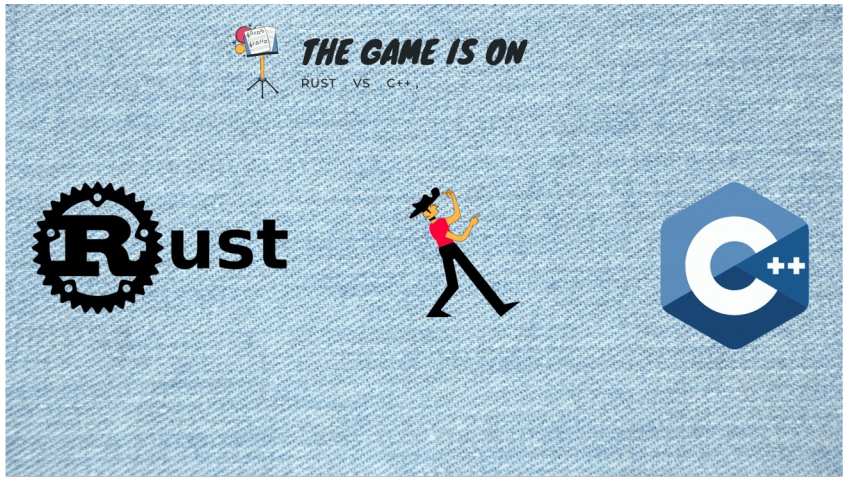


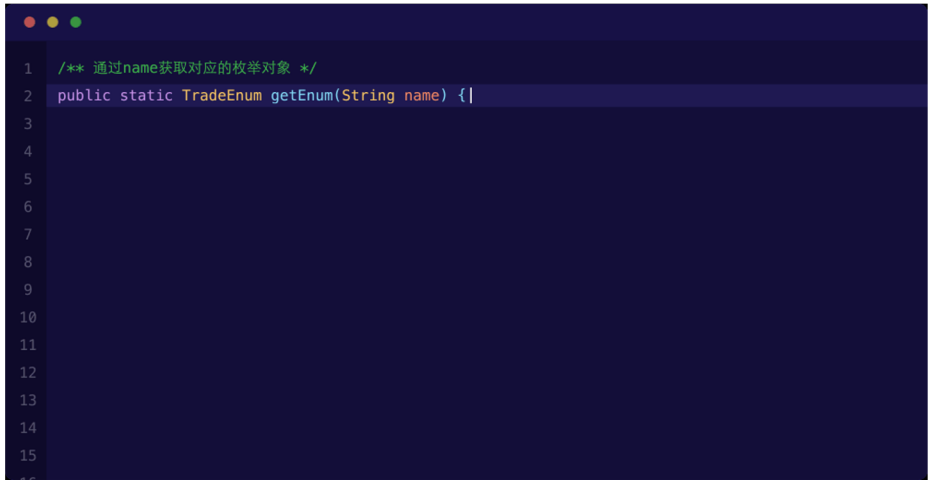
Falsifying AI4Control Software via
Proxy Program Synthesis
(TOSEM 2025)

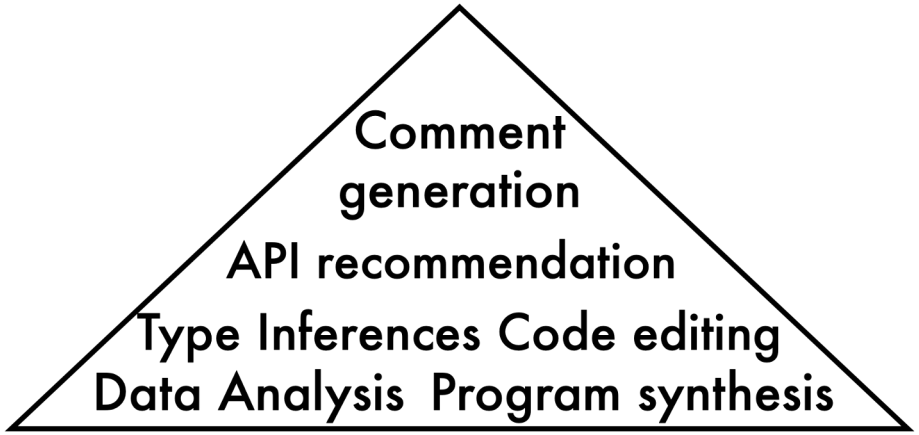
Fortifying AI4Control Software via
Runtime Shield Synthesis
(TOSEM 2025)

Code Large Language Models (CodeLLMs)

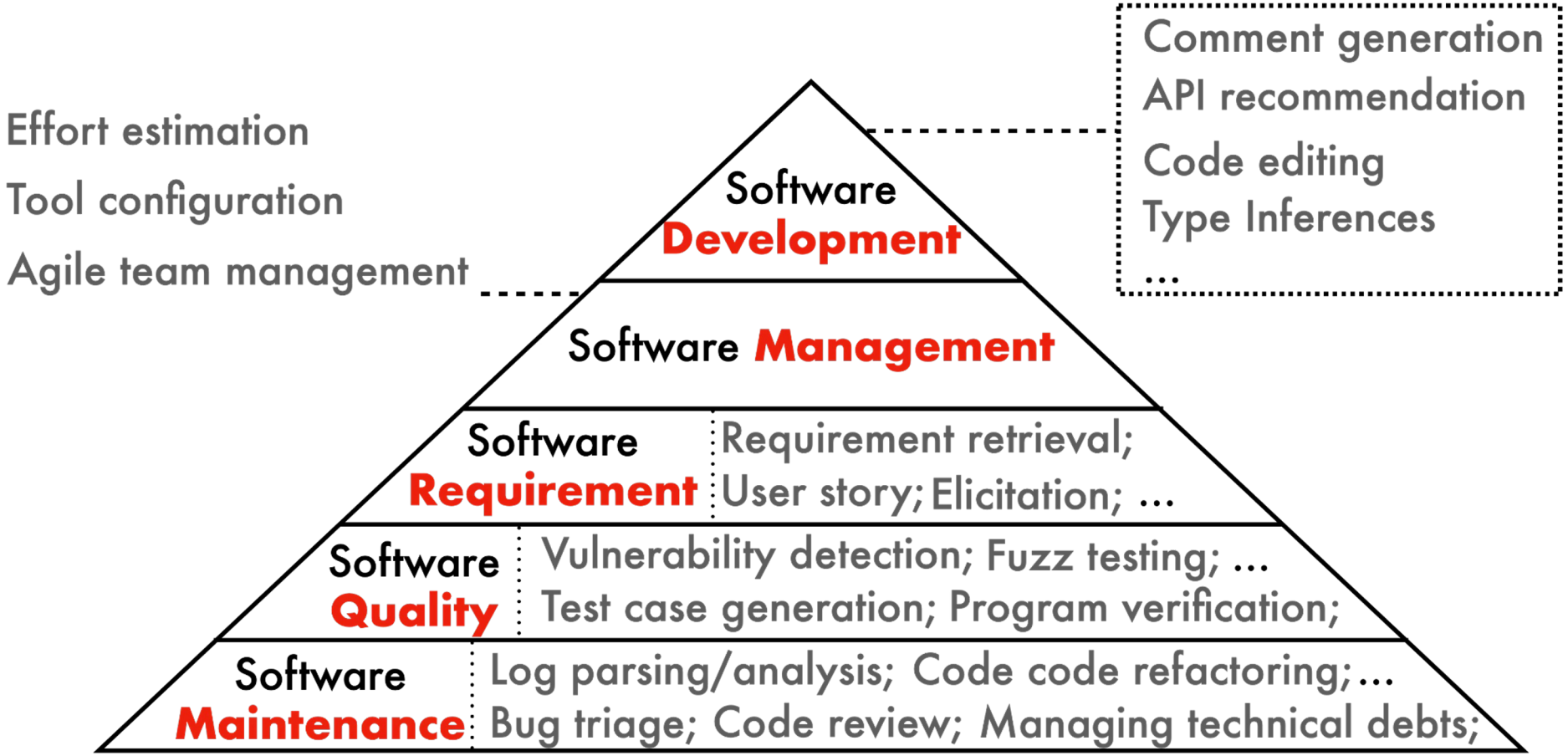






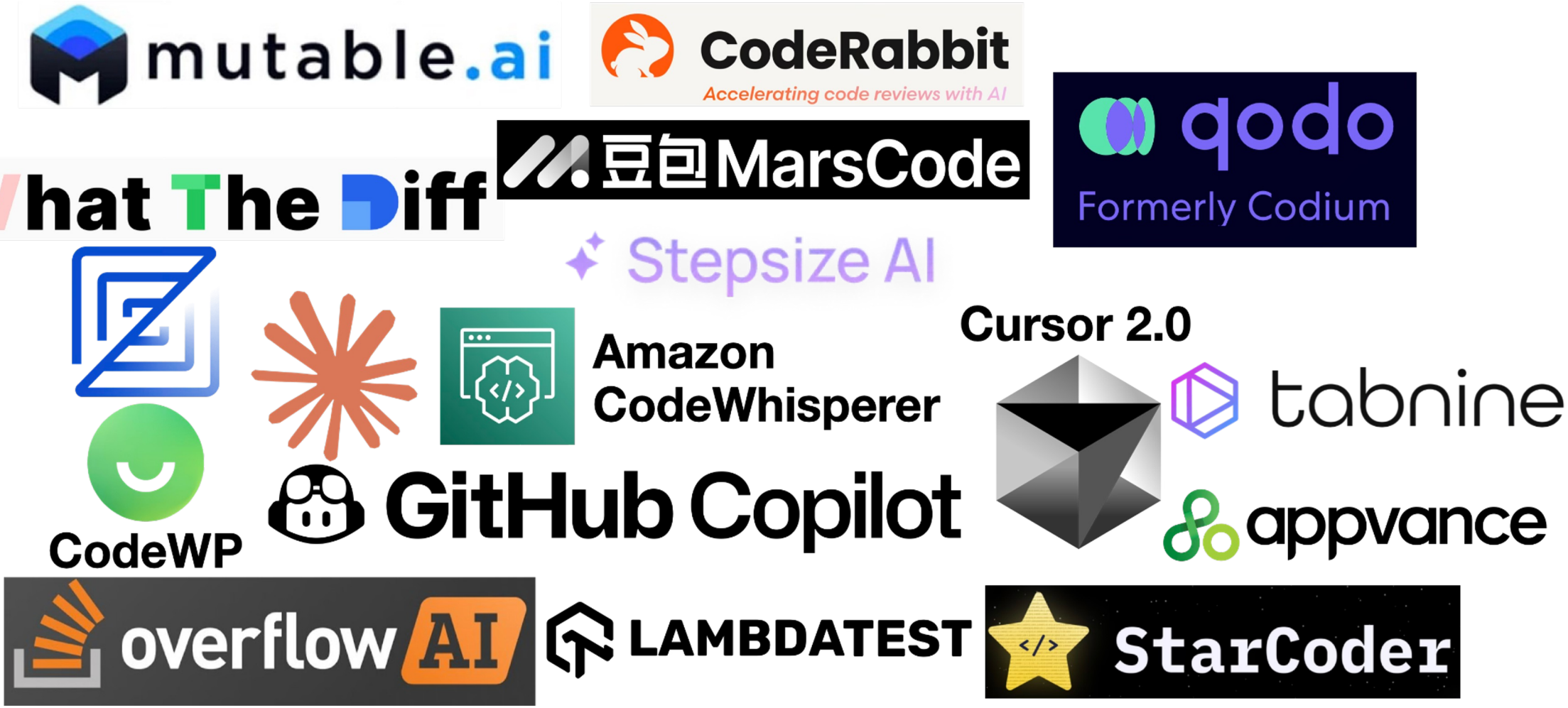


Code Large Language Models (CodeLLMs)



A systematic literature review of AI4SE **[TOSEM-2024]**

Code Large Language Models (CodeLLMs)



What AI coding tools do developers (dis)like?

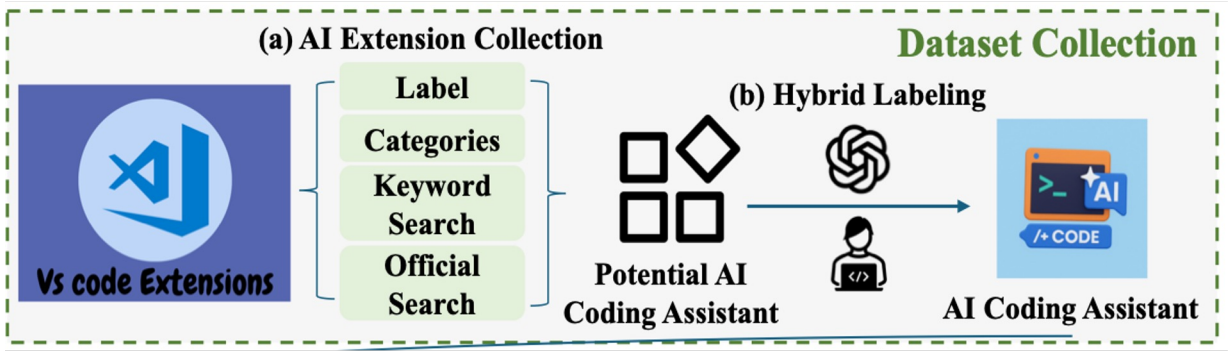
ASE 2025

"*My productivity is boosted, but ...*" Demystifying Users' Perception on AI Coding Assistants

Yunbo Lyu[†], Zhou Yang[‡*], Jieke Shi[†], Jianming Chang[§], Yue Liu[†], David Lo[†]
[†]Singapore Management University [‡]University of Alberta [§]Southeast University
Email: {yunbolyu, jiekeshi, liuyue, davidlo}@smu.edu.sg, zy25@ualberta.ca, jianmingchang@seu.edu.cn

ACM SIGSOFT Distinguished Paper Award

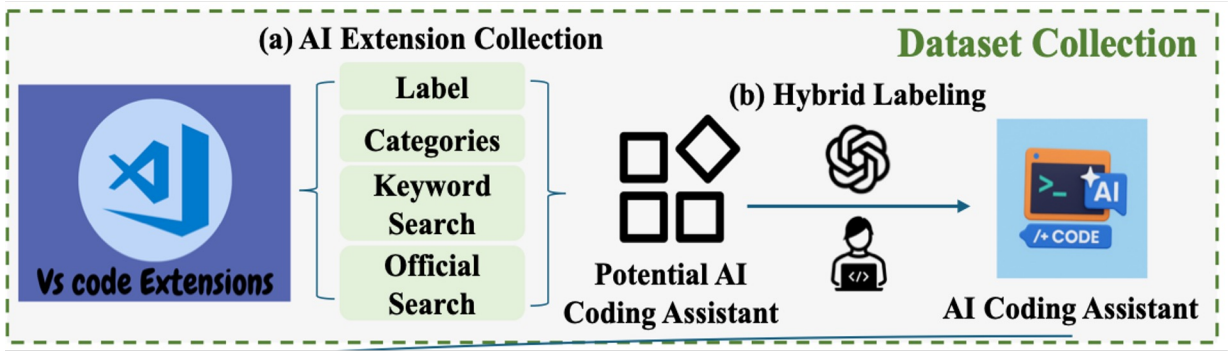
What AI coding tools do developers (dis)like?



Step 1 Identify AI Coding Assistants from VS Code

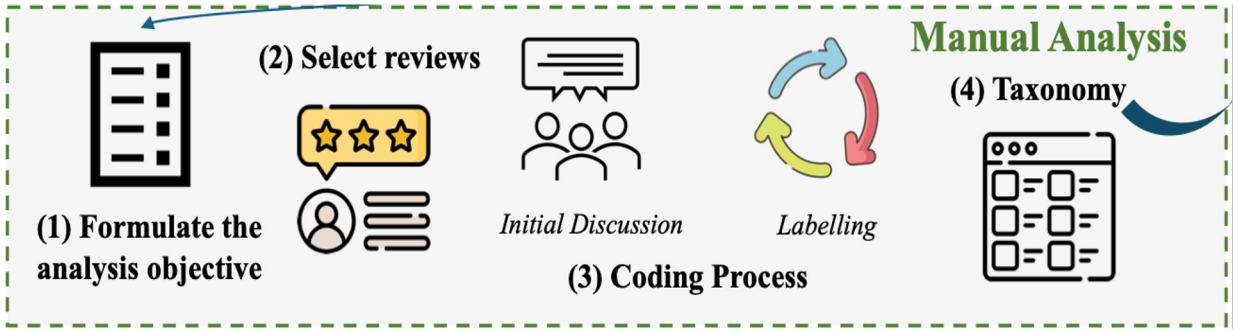
1,085 AI coding assistants identified among over 66K extensions

What AI coding tools do developers (dis)like?



Step I Identify AI Coding Assistants from VS Code

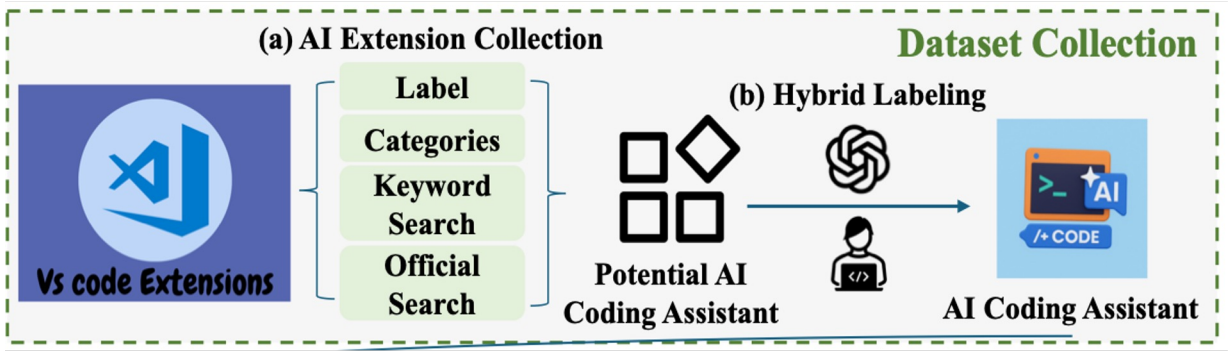
1,085 AI coding assistants identified among over 66K extensions



Step II Manual Analysis User Comments

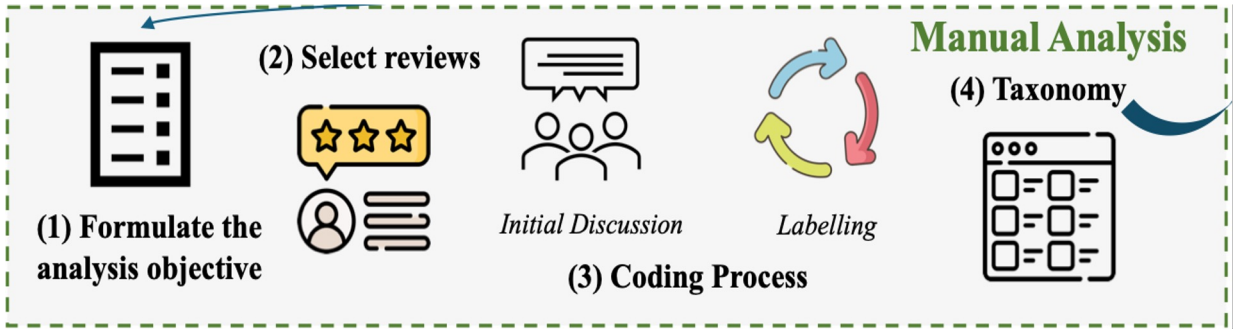
Hybrid card sorting with iterative coding on 361 sampled reviews

What AI coding tools do developers (dis)like?



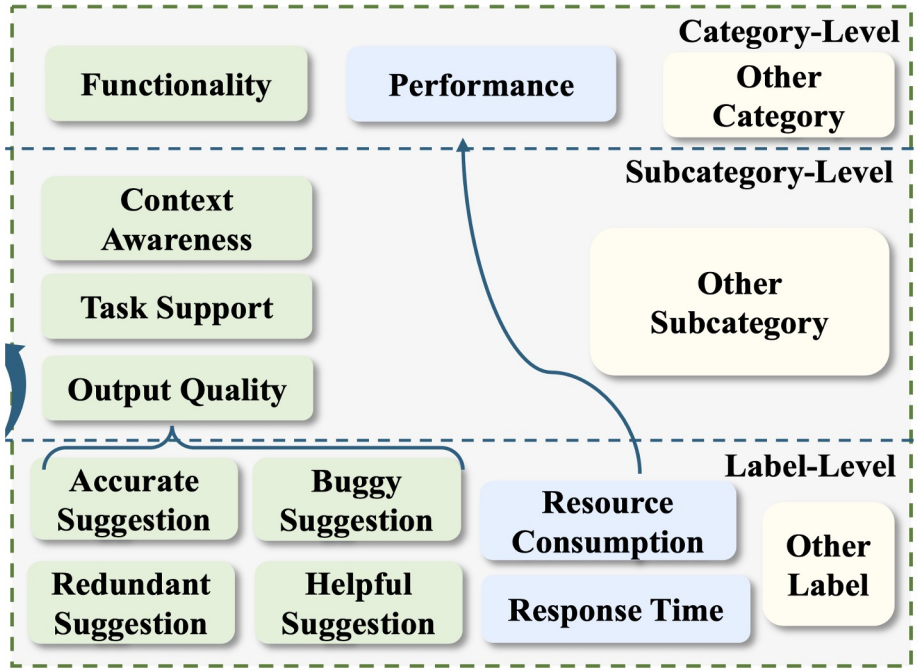
Step I **Identify AI Coding Assistants from VS Code**

1,085 AI coding assistants identified among over 66K extensions



Step II **Manual Analysis User Comments**

Hybrid card sorting with iterative coding on 361 sampled reviews



Step III **Built a 3-level taxonomy**

8 categories, 16 subcategories, 62 labels

Each label for a review is annotated with sentiment (positive, negative, or neutral).

Resource Consumption is a Pain Point

8	**Performance**	Latency, throughput, resource footprint issues.	31	4.2%	58%	42%
	• Response Time	Perceived waiting time between request and output.	17	2.3%	82%	18%
	• Resource Consumption	CPU, memory, battery drain complaints.	9	1.2%	22%	78%
	• Rate Limiting	Complaints about request caps	5	0.7%	40%	60%

Like Request Dislike

Users are dissatisfied with resource consumption but generally satisfied with response time.

Resource Consumption is a Pain Point

8	**Performance**	Latency, throughput, resource footprint issues.	31	4.2%	58%		42%
	• Response Time	Perceived waiting time between request and output.	17	2.3%	82%		18%
	• Resource Consumption	CPU, memory, battery drain complaints.	9	1.2%	22%		78%
	• Rate Limiting	Complaints about request caps	5	0.7%	40%		60%

Users are dissatisfied with resource consumption but generally satisfied with response time.

Uses too many resources—over 50% CPU and more than 1 GB memory" (R125, 1☆).

"The extension's performance can sometimes slow down the editor, especially when working on larger files or multi-projects" (R306, 1☆).

Resource Consumption is a Pain Point

8	**Performance**	Latency, throughput, resource footprint issues.	31	4.2%	58%	42%
	• Response Time	Perceived waiting time between request and output.	17	2.3%	82%	18%
	• Resource Consumption	CPU, memory, battery drain complaints.	9	1.2%	22%	78%
	• Rate Limiting	Complaints about request caps	5	0.7%	40%	60%

Like Request Dislike

Users are dissatisfied with resource consumption but generally satisfied with response time.

Demand for Local and Efficient Deployment

Offline Capability: Users explicitly "complain or request that the extension should work locally (5x) without internet connection".

Model Choice: Users want control, including the "ability to select different models, including local models".

Privacy & Security: Local models avoid sending code to external servers, a concern related to privacy and data leakage.

Demand for Local and Efficient Deployment

Fast and Memory-Efficient Neural Code Completion

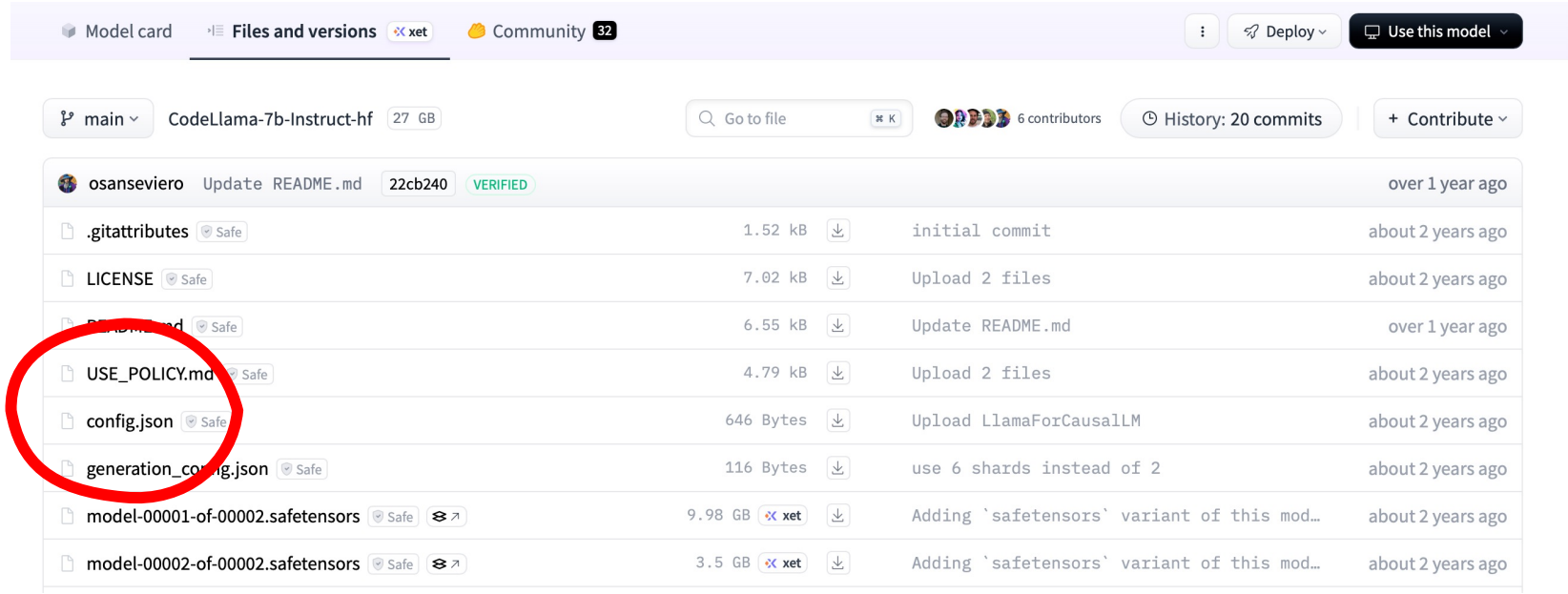
Alexey Svyatkovskiy*, Sebastian Lee†◇, Anna Hadjitofi‡◇,
Maik Riechert*, Juliana Vicente Franco*, Miltiadis Allamanis*
*Microsoft

Small Models. "A **3 MB** model can be deployed even in severely restricted environments (e.g. low-end hardware used to teach students to code). A **50 MB** model is a reasonable upper bound size for a plugin in a modern IDE or editor."

Fast Models. "computational budget should be at most a few tens of milliseconds even on relatively old machines"

-- Svyatkovskiy et al.

Our Approach in a Nutshell: *Hyper-paras./Config. Synthesis*




```json
{
  "tokenizer": "Byte-Pair Encoding",
  "vocab_size": 50265,
  "num_hidden_layers": 12,
  "hidden_size": 768,
  "hidden_act": "GELU",
  "hidden_dropout_prob": 0.1,
  "intermediate_size": 3072,
  "num_attention_heads": 12,
  "attention_probs_dropout_prob": 0.1,
  "max_sequence_length": 512,
  "position_embedding_type": "absolute",
  "learning_rate": 1e-4,
  "batch_size": 32
}
```

Search for the *most appropriate hyper-parameters or configuration*, which yields a compact model with strong performance

Our Approach in a Nutshell: *Hyper-paras./Config. Synthesis*

ASE 2022

Compressing Pre-trained Models of Code into 3 MB

Jieke Shi, Zhou Yang, Bowen Xu*, Hong Jin Kang and David Lo
School of Computing and Information Systems
Singapore Management University
{jiekeshi,zyang,bowenxu.2017,hjkang.2018,davidlo}@smu.edu.sg

First work to compress code LLMs: **160× smaller** and **4.23× faster**

Nominated for ACM SIGSOFT Distinguished Paper Award

ICSE 2024

Greening Large Language Models of Code

Jieke Shi[◇], Zhou Yang[◇], Hong Jin Kang[♠], Bowen Xu[♣], Junda He[◇], and David Lo[◇]
[◇]School of Computing and Information Systems, Singapore Management University, Singapore
[♠]Department of Computer Science, University of California, Los Angeles, USA
[♣]Department of Computer Science, North Carolina State University, Raleigh, USA
{jiekeshi, zyang, jundahe, davidlo}@smu.edu.sg, hjkang@cs.ucla.edu, bxu22@ncsu.edu

Greening Large Language Models of Code

The 46th IEEE/ACM International Conference on Software Engineering (ICSE '24)

Most cited in Software Engineering in Society track by now






Jieke Shi Zhou Yang Hong Jin Kang Bowen Xu Junda He David Lo

SMU SINGAPORE MANAGEMENT UNIVERSITY

UNIVERSITY OF ALBERTA

THE UNIVERSITY OF SYDNEY

NC STATE UNIVERSITY

Homepage

Preprint

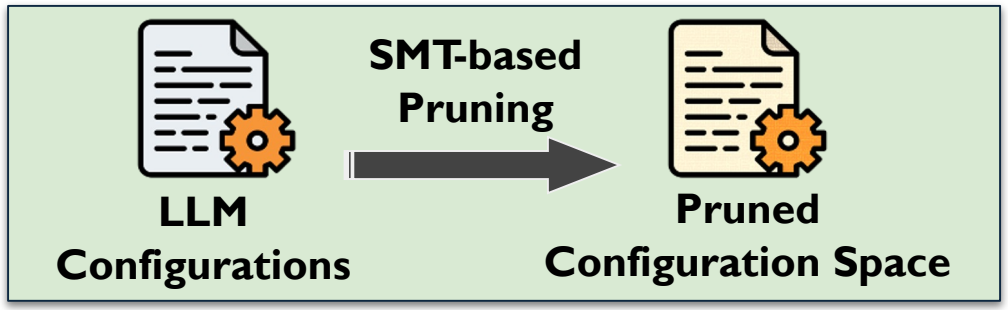
Avatar's Overall Workflow

Avatar's CodeLLMs: **160x smaller**, **76× faster**, **184× more energy-saving**, and **157× less carbon footprint**

Avatar's Overall Workflow

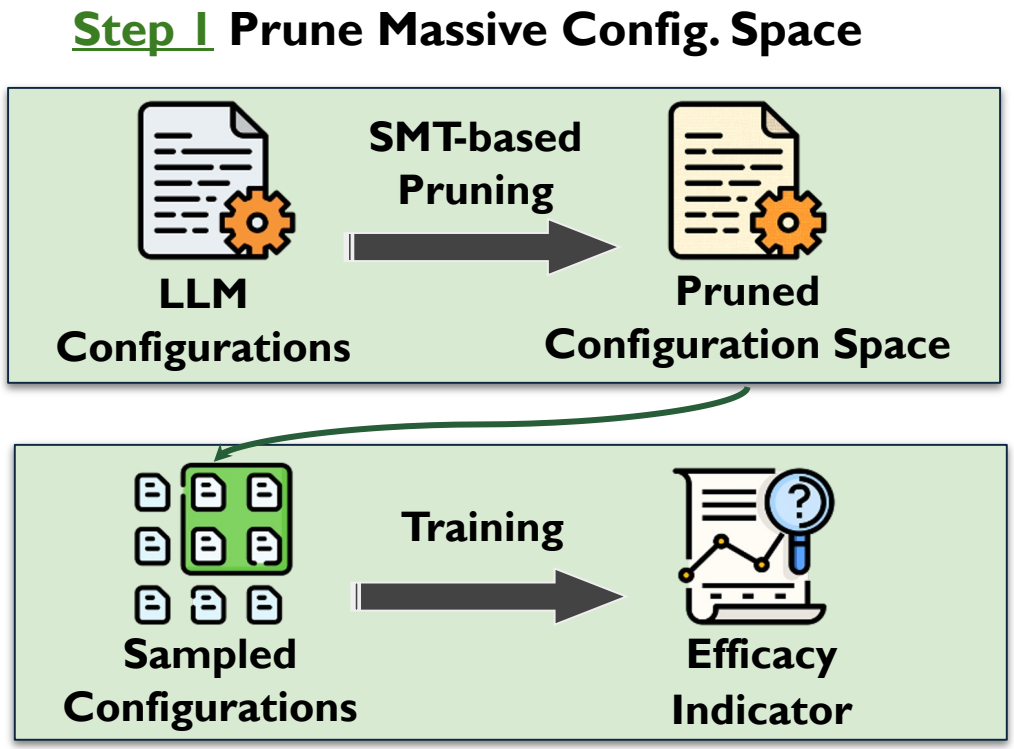
Avatar's CodeLLMs: **160x smaller**, **76× faster**, **184× more energy-saving**, and **157× less carbon footprint**

Step 1 **Prune Massive Config. Space**



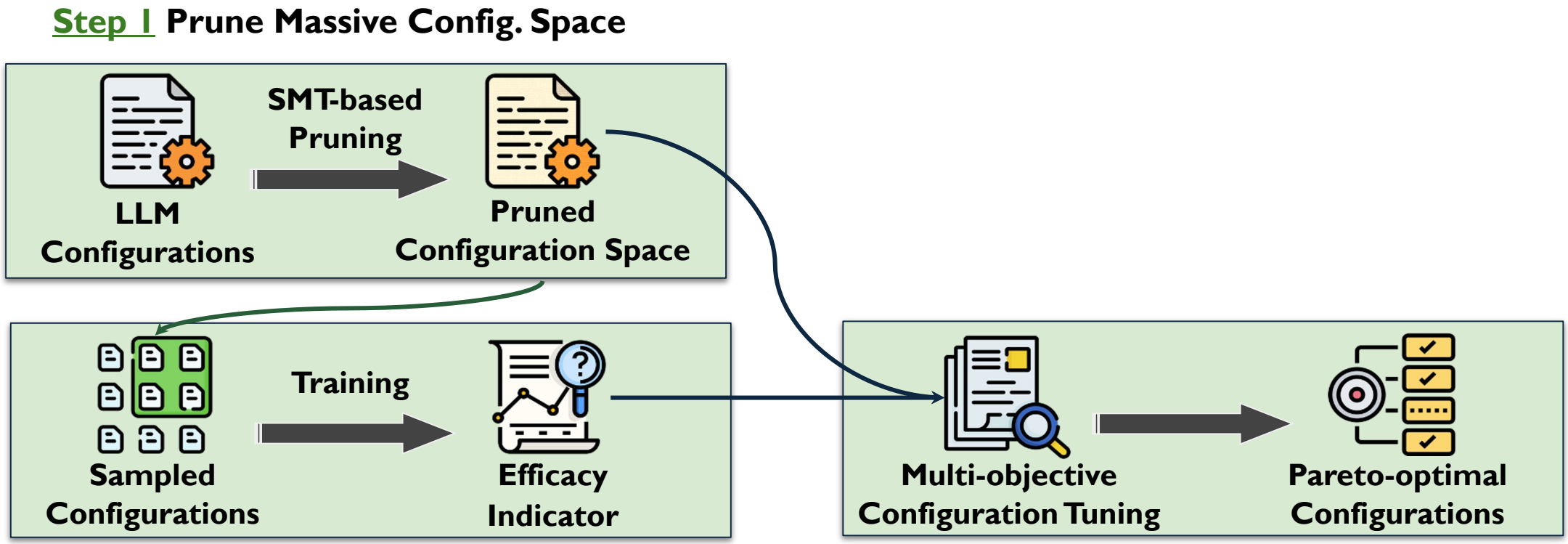
Avatar's Overall Workflow

Avatar's CodeLLMs: **160x smaller**, **76× faster**, **184× more energy-saving**, and **157× less carbon footprint**



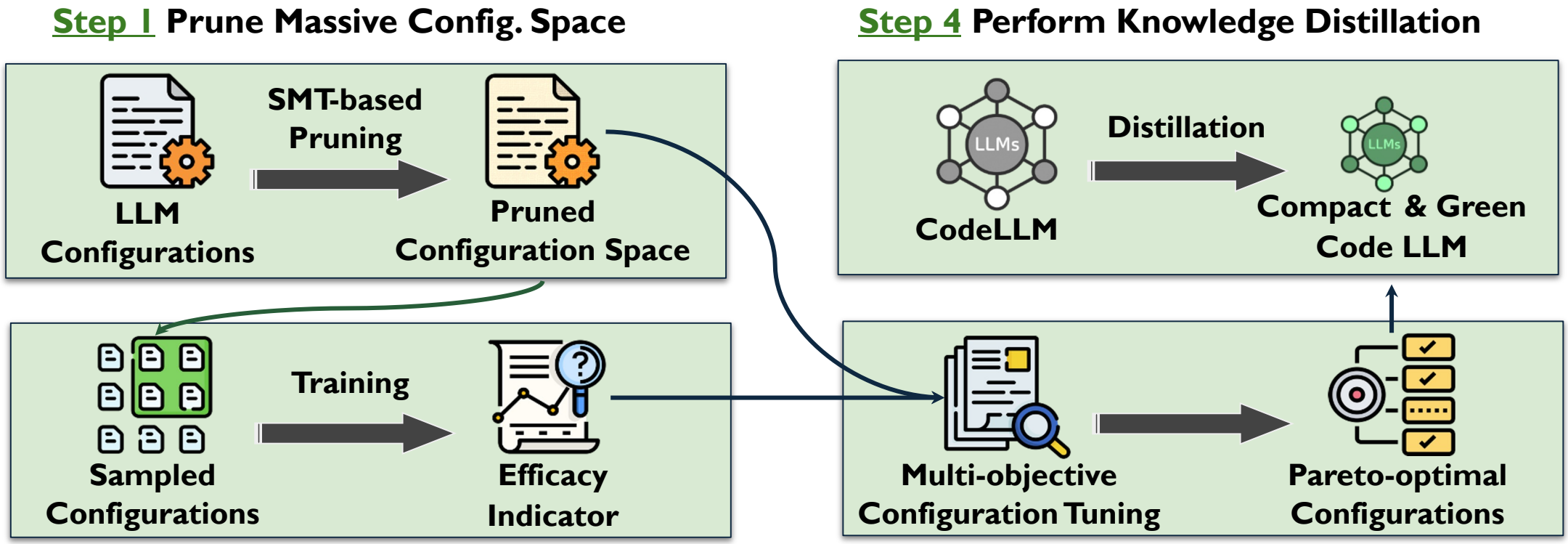
Avatar's Overall Workflow

Avatar's CodeLLMs: **160x smaller**, **76× faster**, **184× more energy-saving**, and **157× less carbon footprint**



Avatar's Overall Workflow

Avatar's CodeLLMs: **160x smaller**, **76× faster**, **184× more energy-saving**, and **157× less carbon footprint**



Step 1: Prune Massive Configuration Space

```
"tokenizer": ["Byte-Pair Encoding", "WordPiece",
   ↪ "Unigram", "Word"],
"vocab_size": range(1000, 50265),
"num_hidden_layers": range(1, 12),
"hidden_size": range(16, 768),
"hidden_act": ["GELU", "ReLU", "SiLU", "GELU_new"],
"hidden_dropout_prob": [0.1, 0.2, 0.3, 0.4, 0.5],
"intermediate_size": range(16, 3072),
"num_attention_heads": range(1, 12),
"attention_probs_dropout_prob": [0.1, 0.2, 0.3, 0.4,
   ↪0.5],
"max_sequence_length": range(256, 512),
"position_embedding_type":["absolute", "relative_key",
   ↪ "relative_key_query"],
"learning_rate": [1e-3, 1e-4, 5e-5],
"batch_size": [16, 32, 64]
```

Typical configuration space of CodeLLMs containing

4.5×10^{19} plausible configurations

Too large & many are infeasible!

Step 1: Prune Massive Configuration Space

Let Formal Constraint Solvers help!

Step 1: Prune Massive Configuration Space

Let Formal Constraint Solvers help!

Formulating model size and its constraint:

$$\texttt{size}(c) \leq 3\ \text{MB} \quad \text{s.t.} \quad c \in C$$

$$\texttt{size}(c) = \frac{4(v+s+3)h}{1024 \times 1024} + \frac{4(4h^2 + (9+2i)h + i)l}{1024 \times 1024} + \frac{2h^2 + 4h + 2}{1024 \times 1024}$$

- *C:* the configuration space
- *c:* a configuration
- *v:* vocabulary size
- *s:* maximum input length
- *l:* number of hidden layers
- *h:* hidden layer dimension
- *i:* intermediate NN layer dimension

Step 1: Prune Massive Configuration Space

Let Formal Constraint Solvers help!

Formulating model size and its constraint:

$$\texttt{size}(c) \leq 3\ \text{MB} \quad \text{s.t.} \quad c \in C$$

$$\texttt{size}(c) = \frac{4(v+s+3)h}{1024 \times 1024} + \frac{4(4h^2+(9+2i)h+i)l}{1024 \times 1024} + \frac{2h^2+4h+2}{1024 \times 1024}$$

- *C:* the configuration space
- *c:* a configuration
- *v:* vocabulary size
- *s:* maximum input length
- *l:* number of hidden layers
- *h:* hidden layer dimension
- *i:* intermediate NN layer dimension

Configuration feasibility is verified using Microsoft's SMT solver Z3

Z3 Microsoft Research

encodes config. as logical constraints & checks if they're satisfiable

```
;; Check if a model configuration satisfies memory constraint
(declare-const v Int)  ; vocabulary size
(declare-const h Int)  ; hidden dim
(declare-const s Int)  ; max input length
(assert (and (> v 1000) (< v 5000)))
(assert (<= (+ (* 4 h s) (* v h)) 3_000_000)) ; <= 3 MB
(check-sat)
(get-model)
......
```

Step 1: Prune Massive Configuration Space

Let Formal Constraint Solvers help!

Formulating model size and its constraint:

$$\texttt{size}(c) \le 3\ \text{MB} \quad \text{s.t.} \quad c \in C$$

$$\texttt{size}(c) = \frac{4(v+s+3)h}{1024\times 1024} + \frac{4(4h^2+(9+2i)h+i)l}{1024\times 1024} + \frac{2h^2+4h+2}{1024\times 1024}$$

- *C:* the configuration space
- *c:* a configuration
- *v:* vocabulary size
- *s:* maximum input length
- *l:* number of hidden layers
- *h:* hidden layer dimension
- *i:* intermediate NN layer dimension

Configuration feasibility is verified using Microsoft's SMT solver Z3

Z3 Microsoft Research

encodes config. as logical constraints & checks if they're satisfiable

```
;; Check if a model configuration satisfies memory constraint
(declare-const v Int)  ; vocabulary size
(declare-const h Int)  ; hidden dim
(declare-const s Int)  ; max input length
(assert (and (> v 1000) (< v 5000)))
(assert (<= (+ (* 4 h s) (* v h)) 3_000_000)) ; <= 3 MB
(check-sat)
(get-model)
......
```

Remaining space after pruning accounts for **only 28.9%** of the original one

Step 3: Identify Pareto-Optimal Configurations

Avatar uses a *multi-objective optimization* algorithm to find

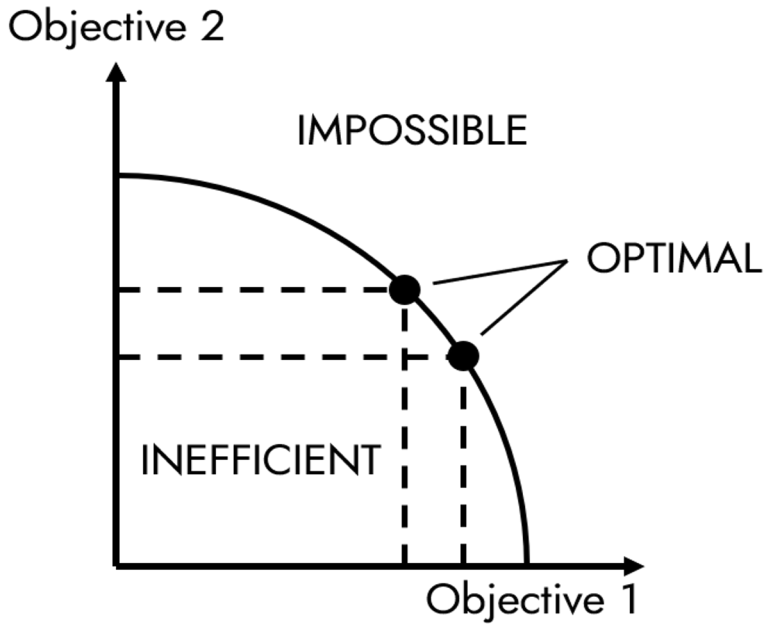
Pareto-optimal configurations

Step 3: Identify Pareto-Optimal Configurations

Avatar uses a *multi-objective optimization* algorithm to find

Pareto-optimal configurations

configurations that achieve the **best trade-off among all objectives**

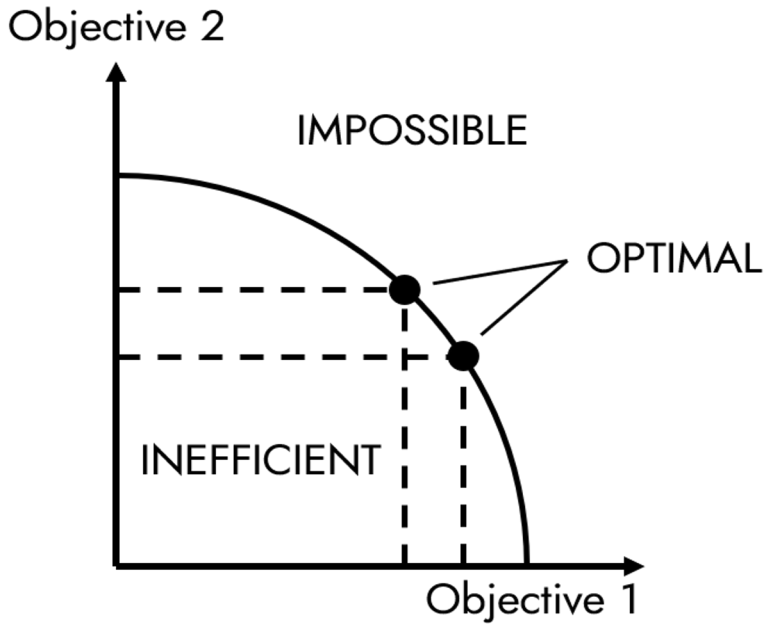


Step 3: Identify Pareto-Optimal Configurations

Avatar uses a *multi-objective optimization* algorithm to find

Pareto-optimal configurations

configurations that achieve the **best trade-off among all objectives**

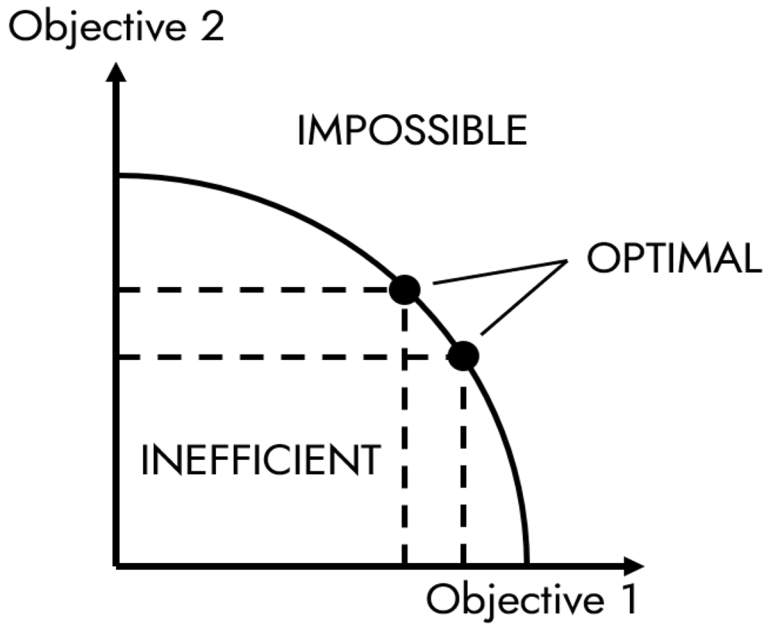


$$\min_{c} \quad \{\texttt{size}(c), \texttt{FLOPs}(c), -\texttt{effectiveness}(c)\}$$
$$\text{s.t.} \quad c \in C$$

Step 3: Identify Pareto-Optimal Configurations

Avatar uses a *multi-objective optimization* algorithm to find **Pareto-optimal configurations**

configurations that achieve the **best trade-off among all objectives**



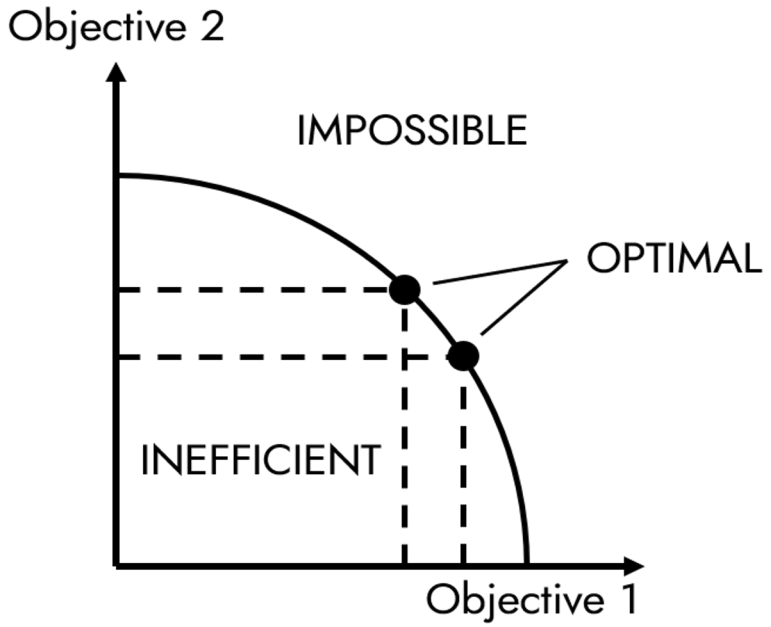
Model Size

$$\min_{c} \quad \{\texttt{size}(c), \texttt{FLOPs}(c), -\texttt{effectiveness}(c)\}$$
$$\text{s.t.} \quad c \in C$$

Step 3: Identify Pareto-Optimal Configurations

Avatar uses a *multi-objective optimization* algorithm to find **Pareto-optimal configurations**

configurations that achieve the **best trade-off among all objectives**



Model Size

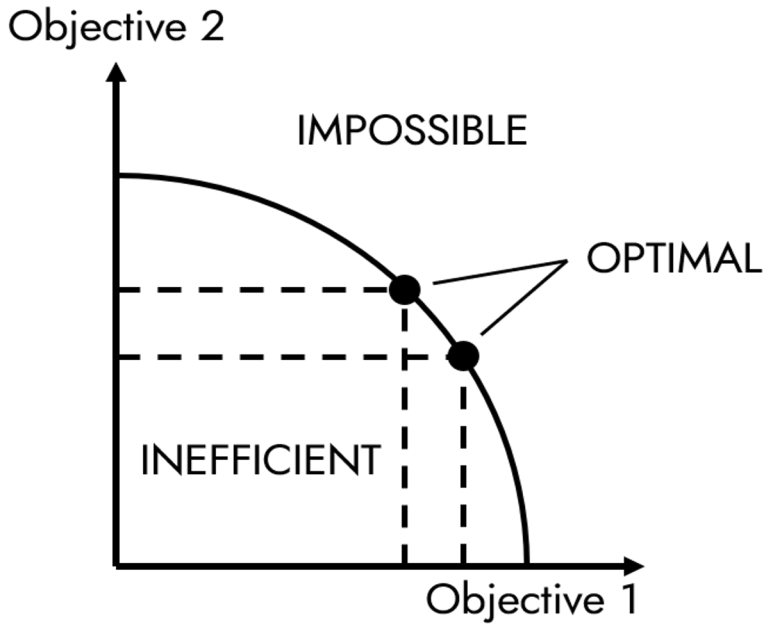
$$\min_{c} \quad \{\texttt{size}(c), \texttt{FLOPs}(c), -\texttt{effectiveness}(c)\}$$
$$\text{s.t.} \quad c \in C$$

Pruned Config. Space from Step 1

Step 3: Identify Pareto-Optimal Configurations

Avatar uses a *multi-objective optimization* algorithm to find **Pareto-optimal configurations**

configurations that achieve the **best trade-off among all objectives**



Model Size

$$\min_{c} \{\texttt{size}(c), \texttt{FLOPs}(c), -\texttt{effectiveness}(c)\}$$
$$\text{s.t.} \quad c \in C$$

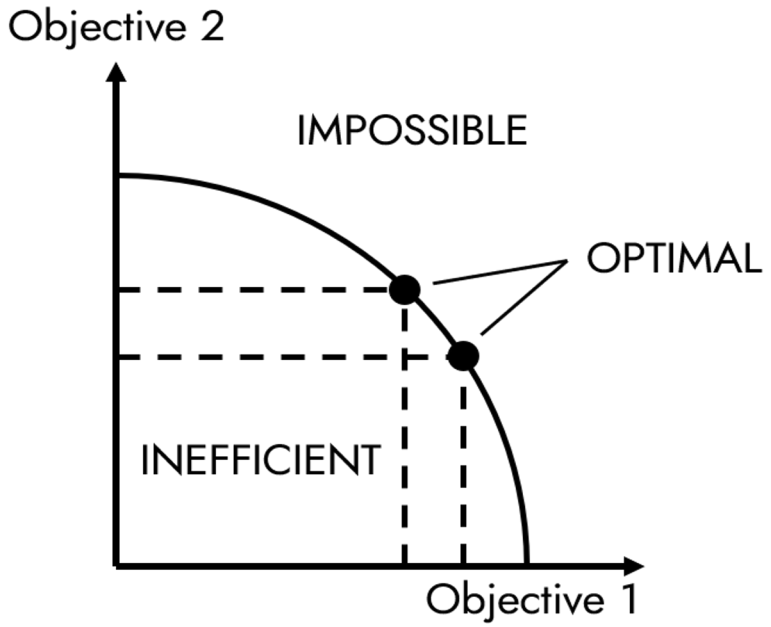
Pruned Config. Space from Step 1

Proxy of CodeLLM's Inference Latency, Energy Consumption & Carbon Footprint

Step 3: Identify Pareto-Optimal Configurations

Avatar uses a *multi-objective optimization* algorithm to find **Pareto-optimal configurations**

configurations that achieve the **best trade-off among all objectives**



Model Size

Prediction Accuracy on downstream task (using the efficacy indicator created in Step 2)

$$\min_{c} \{\texttt{size}(c), \texttt{FLOPs}(c), -\texttt{effectiveness}(c)\}$$
$$\text{s.t.} \quad c \in C$$

Pruned Config. Space from Step 1

Proxy of CodeLLM's Inference Latency, Energy Consumption & Carbon Footprint

School of Computing and Information Systems

SMU SINGAPORE MANAGEMENT UNIVERSITY

Step 3: Identify Pareto-Optimal Configurations

Avatar uses a *multi-objective optimization* algorithm to find

Pareto-optimal configurations

configurations that achieve the **best trade-off amon**

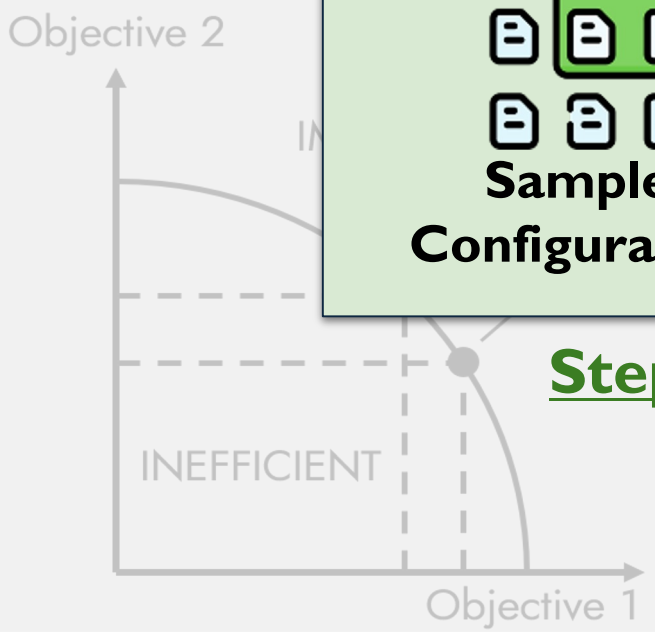
Step 2 Build Efficacy Indicator

Prediction Accuracy on
m task (using the
icator created in Step 2)

ffectiveness(c)}

Proxy of CodeLLM's Inference Latency, Energy Consumption & Carbon Footprint

Pruned Config. Space from Step 1



School of Computing and Information Systems

SMU SINGAPORE MANAGEMENT UNIVERSITY

Step 4: Perform Knowledge Distillation

$$\mathcal{L} = -\frac{1}{n}\sum_{i}^{n}\sum_{j}^{z} softmax(\frac{p_{ij}}{T}) \log\left(softmax(\frac{q_{ij}}{T})\right) T^2$$

Step 4: Perform Knowledge Distillation

$$\mathcal{L} = -\frac{1}{n}\sum_{i}^{n}\sum_{j}^{z} softmax(\frac{p_{ij}}{T}) \log\left(softmax(\frac{q_{ij}}{T})\right) T^2$$

Num of training examples

Num of classes

Softmax function

temperature parameter

Step 4: Perform Knowledge Distillation

Outputs of the large CodeLLM and small model being trained, respectively

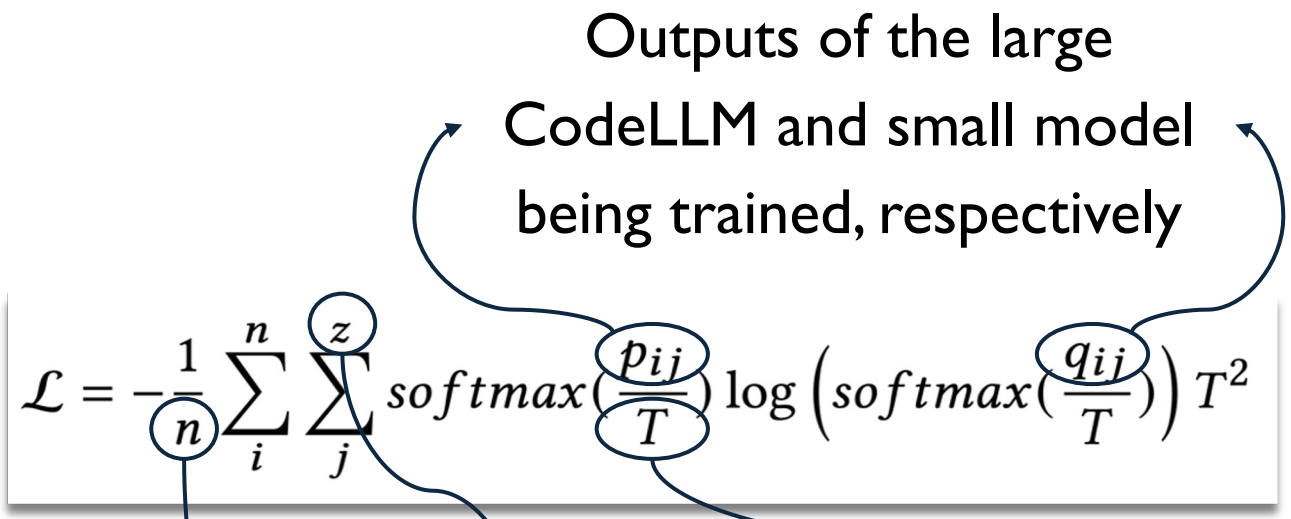
$$\mathcal{L} = -\frac{1}{n}\sum_{i}^{n}\sum_{j}^{z} softmax\left(\frac{p_{ij}}{T}\right) \log\left(softmax\left(\frac{q_{ij}}{T}\right)\right) T^2$$

Num of training examples

Num of classes

Softmax function temperature parameter

Step 4: Perform Knowledge Distillation



Outputs of the large CodeLLM and small model being trained, respectively

Num of classes

Softmax function temperature parameter

Num of training examples

Minimizing this loss means making the outputs of the large and the small CodeLLMs **as similar as possible**

Results: Effectiveness on Various LLMs

Avatar effectively optimizes encoder-based CodeLLMs CodeBERT & GraphCodeBERT on Vulnerability Prediction & Clone Detection in terms of

Results: Effectiveness on Various LLMs

Avatar effectively optimizes encoder-based CodeLLMs CodeBERT & GraphCodeBERT on Vulnerability Prediction & Clone Detection in terms of

model size	inferency latency
481 MB to **3 MB** **160×** smaller	up to **76×** faster
efficacy	**throughput**
only 1.67% loss	up to **9.7x** more queries

Results: Effectiveness on Various LLMs

Avatar effectively optimizes encoder-based CodeLLMs CodeBERT & GraphCodeBERT on Vulnerability Prediction & Clone Detection in terms of

model size	inferency latency	carbon footprint
481 MB to **3 MB** **160×** smaller	up to **76×** faster	up to **157×** less

efficacy	throughput	energy consumption
only 1.67% loss	up to **9.7x** more queries	up to **184×** less

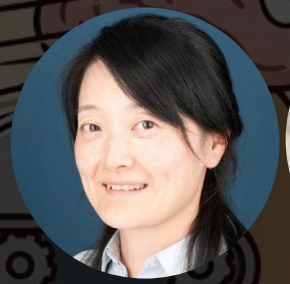
Towards Efficient Code LLM Inference with **Quantization & Compilation-Time Optimization**

under submission









Jieke Shi Junda He Zhou Yang Chengran Yang Mike Klymenko James Hoang Sherry Xu Zhenchang Xing David Lo

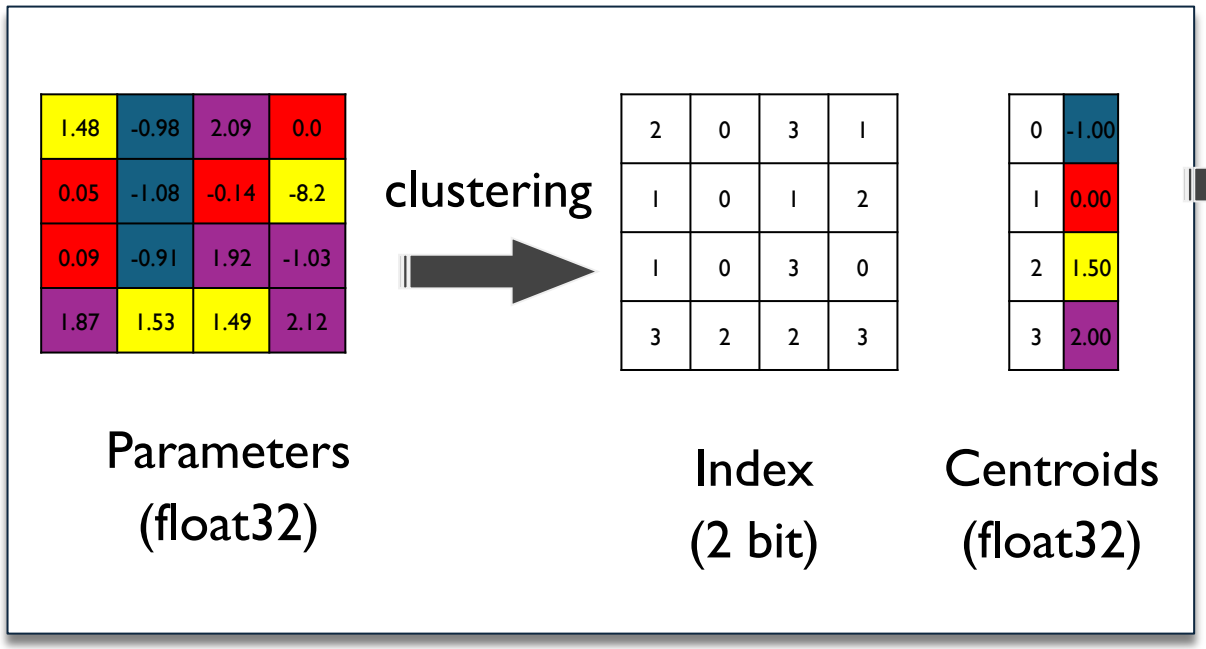
SMU SINGAPORE MANAGEMENT UNIVERSITY

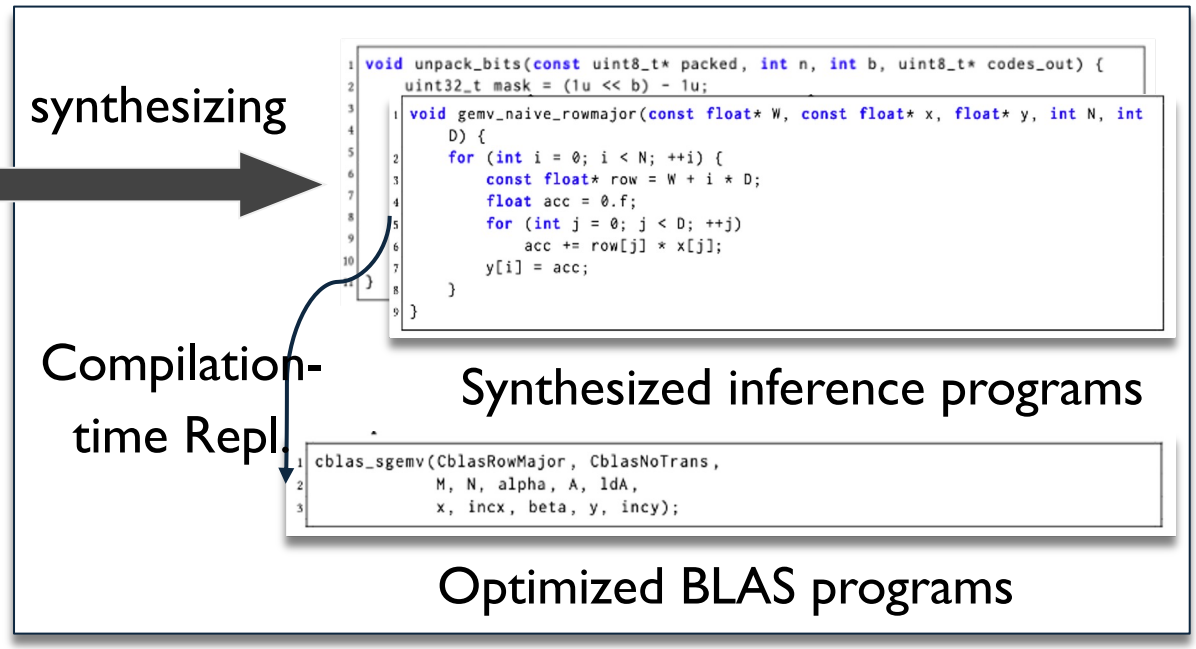
UNIVERSITY OF ALBERTA

CSIRO DATA 61

Ditto's Overall Workflow

Ditto's CodeLLMs: **10.5x faster**, **6.4× lower memory usage**,
and **only 0.27% loss in pass@1**



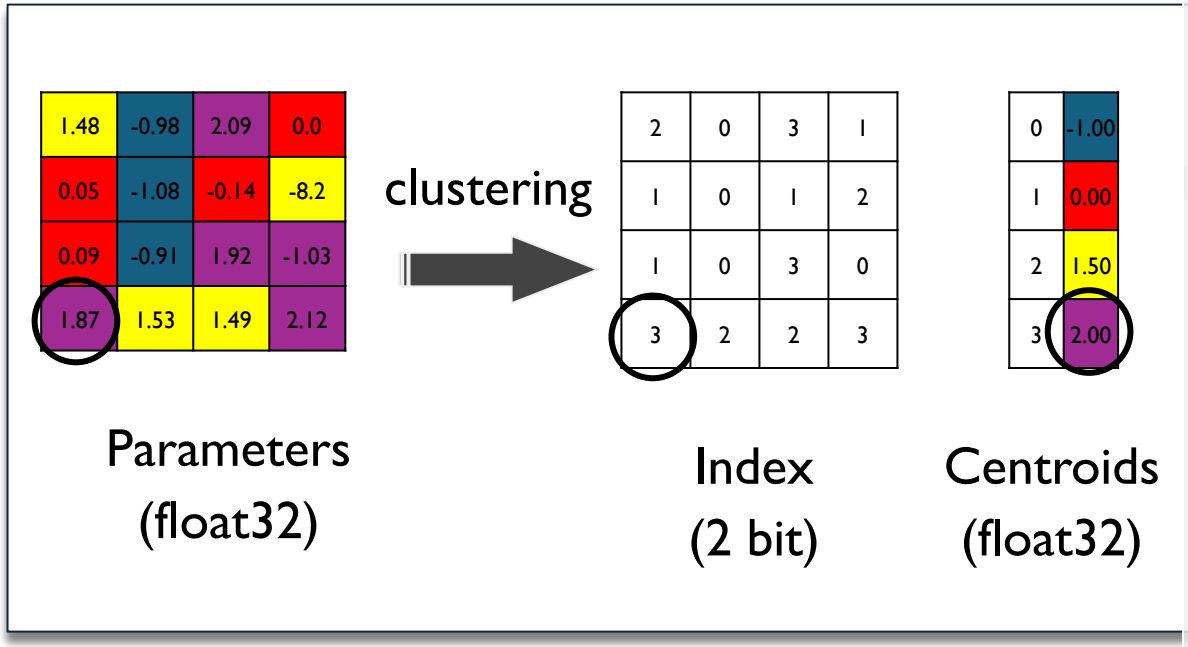


Step 1 Weight Quantization

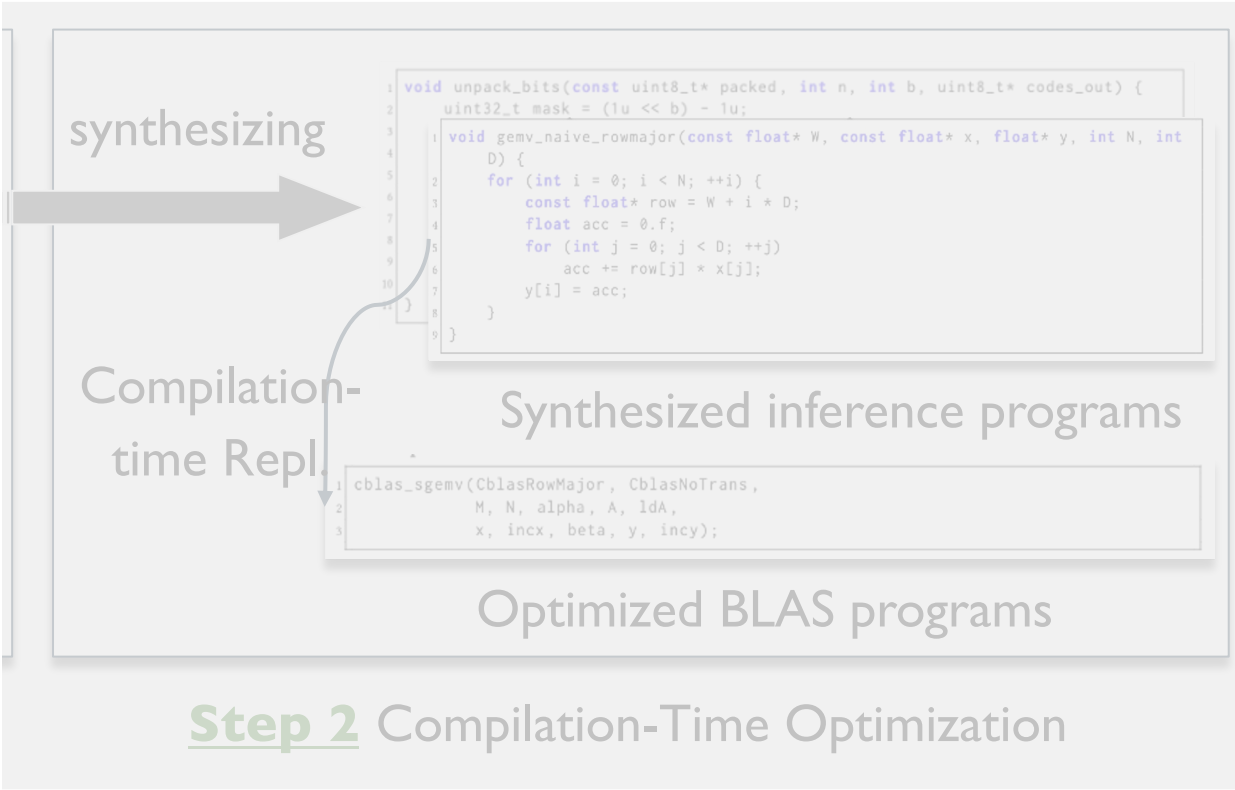
Step 2 Compilation-Time Optimization

Ditto's Overall Workflow

Ditto's CodeLLMs: **10.5x faster, 6.4× lower memory usage,**
and **only 0.27% loss in pass@1**



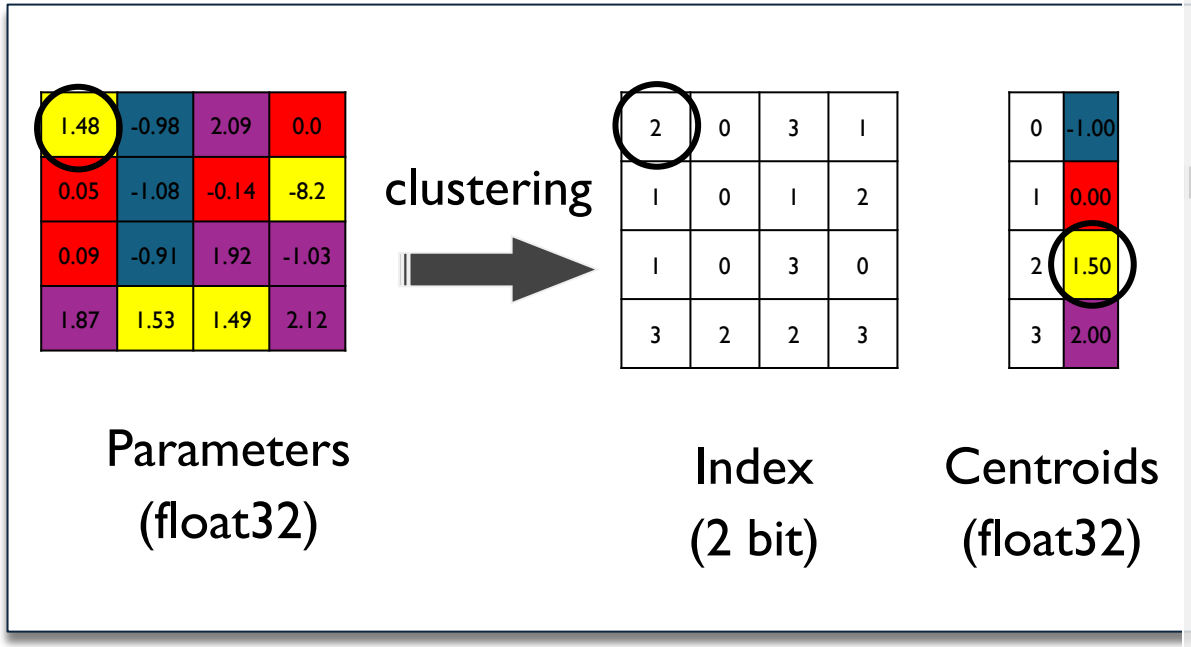
Step 1 Weight Quantization



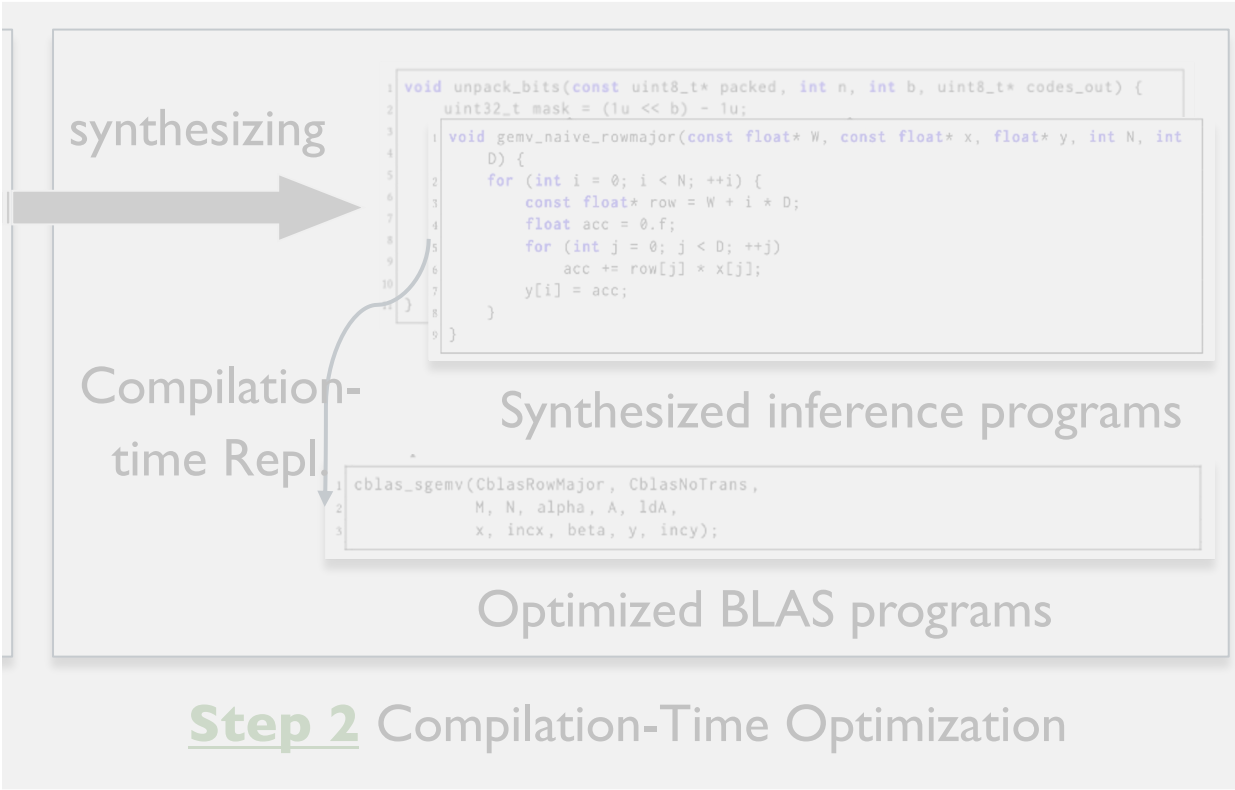
Step 2 Compilation-Time Optimization

Ditto's Overall Workflow

Ditto's CodeLLMs: **10.5x faster, 6.4× lower memory usage,**
and **only 0.27% loss in pass@1**



Step 1 Weight Quantization



Step 2 Compilation-Time Optimization

Step 2: Compilation-Time Optimization

Ditto identifies unoptimized General Matrix–Vector Multiplication (GEMV) operations and replaces them with optimized implementations using **Basic Linear Algebra Subprograms (BLAS) primitives**.

Match Phase

- extends LLVM's PatternMatch API
- identifies nested loops where the inner loop performs multiply–add reductions over affine memory accesses (e.g., y[i] += A[i][j] * x[j])

Legality Check

- Ensures no side effects (e.g., aliasing loads or breaking dependencies)
- Infers whether the matrix is row-major or column-major to ensure safe transformation

Replacement Phase

```
void gemv_naive(const float* A, const float* x, float* y, int m, int n,
    float alpha, float beta) {
    for (int i = 0; i < m; ++i) {
        float sum = 0.0f;
        for (int j = 0; j < n; ++j) {
            sum += A[i * n + j] * x[j];
        }
        y[i] = alpha * sum + beta * y[i];
    }
}
```

```
void cblas_dgemv(
    const CBLAS_LAYOUT Layout,
    const CBLAS_TRANSPOSE trans,
    const int m, const int n,
    const double alpha,
    const double *A, const int lda,
    const double *x, const int incx,
    const double beta, double *y, const int incy
);
```

Results: Effectiveness on Various LLMs

Ditto effectively optimizes decoder-based CodeLLMs Code Llama-7B, OpenCodeInterpreter-CL-7B, & Magicoder-CL-7B on HumanEval+ & MBPP+ benchmarks in terms of

efficacy	inferency latency	memory usage
at most a **1.83%** drop in pass@1, **4.45%** in pass@5	accelerates inference by up to **10.5×**	reduces memory usage by up to **6.4×**

Efficient and Green Large Language Models for Software Engineering: Literature Review, Vision, and the Road Ahead

ACM Transactions on Software Engineering and Methodology (TOSEM 2025)


Jieke Shi


Zhou Yang


David Lo

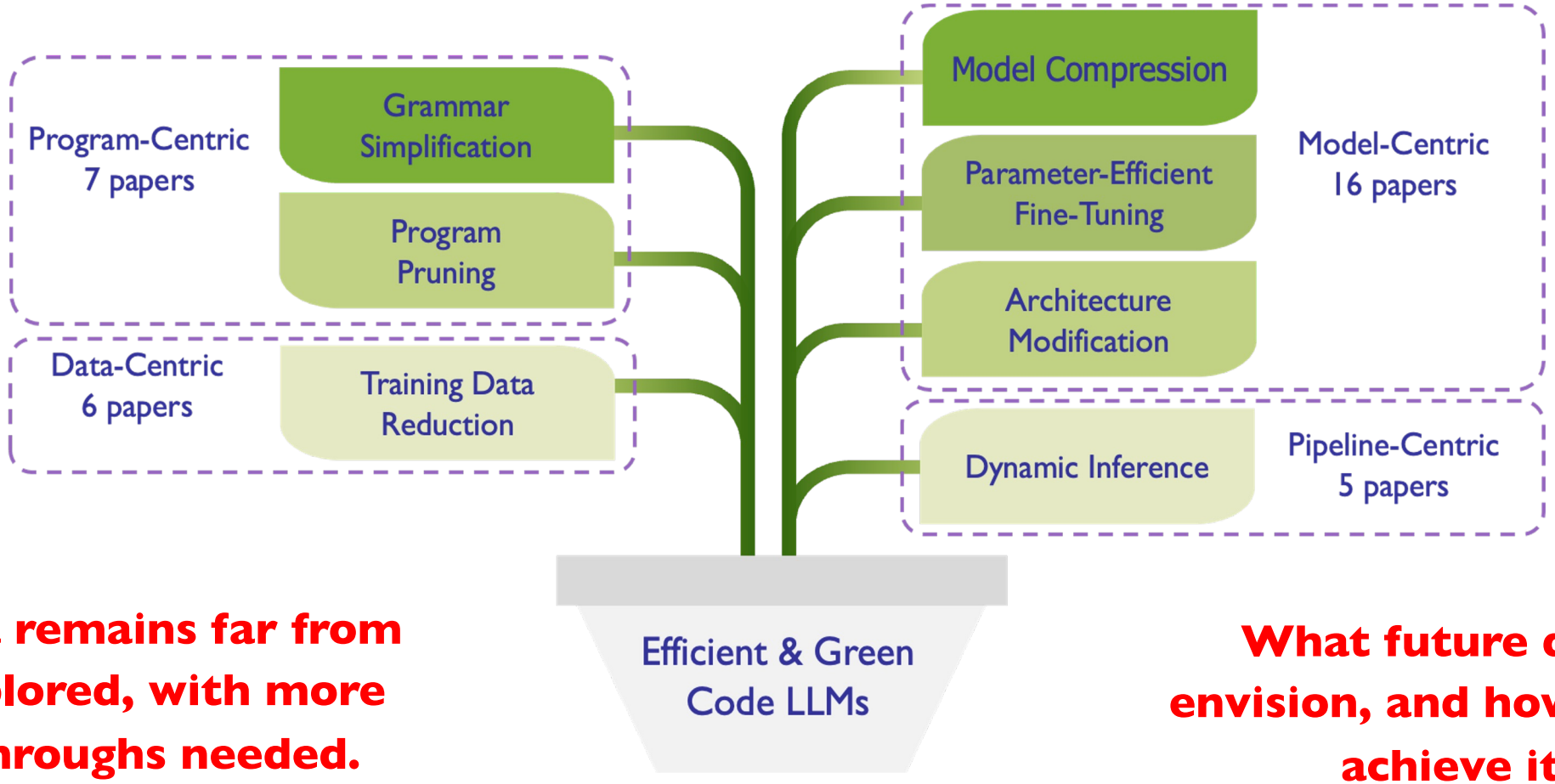
SMU SINGAPORE MANAGEMENT UNIVERSITY

UNIVERSITY OF ALBERTA

Homepage

Preprint

Literature Review



This area remains far from fully explored, with more breakthroughs needed.

What future do we envision, and how can we achieve it?

Why Safer AI4Control Software Matter

AI is now deciding how systems act

Why Safer AI4Control Software Matter

AI is now deciding how systems act

The Mercury News

Waymo robotaxis make up 20% of Uber rides in Austin, data shows

Waymo robotaxis made up about 20% of rides offered by Uber Technologies Inc. in Austin in the last week of March, underscoring how consumers...

2 days ago

"..Industry 4.0 has the potential to create a massive amount of value, generating up to **$3.7 trillion by 2025**…"

-- Microsoft Intelligent Industry Whitepaper

Why Safer AI4Control Software Matter

AI is now deciding how systems act, but also killing people…

The Mercury News

Waymo robotaxis make up 20% of Uber rides in Austin, data shows

Waymo robotaxis made up about 20% of rides offered by Uber Technologies Inc. in Austin in the last week of March, underscoring how consumers...

2 days ago

Telegrafi

Robot kills a worker in South Korea: It mistook him for a food box, crushed his face and chest

Robot kills a worker in South Korea: It mistook him for a food box, crushed his face and chest ... A man has been crushed to death by a robot in South Korea after...

17 Oct 2024

"..Industry 4.0 has the potential to create a massive amount of value, generating up to **$3.7 trillion by 2025**…"

-- Microsoft Intelligent Industry Whitepaper

Ingka Group

IKEA is elevating human – AI-powered drone collaboration

IKEA is thrilled to announce the next phase in its drone technology: an upgraded AI-powered system capable of operating alongside co-workers around the clock.

16 Aug 2024

How Can Software Engineering Help?

How Can Software Engineering Help?

Testing & Analysis — to uncover safety violations

How Can Software Engineering Help?

Testing & Analysis — to uncover safety violations

Healing & Mitigation — to correct or reduce issues dynamically, without retraining

Finding Safety Violations of AI-Enabled Control Systems through the Lens of **Synthesized Proxy Programs**

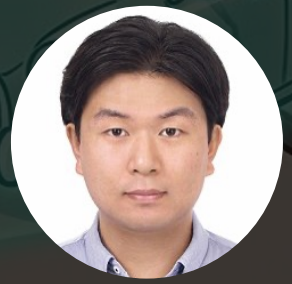
ACM Transactions on Software Engineering and Methodology (TOSEM), 2025

Jieke Shi

Zhou Yang

Junda He

Bowen Xu



Dongsun Kim

Donggyun Han

David Lo

SMU Singapore Management University

University of Alberta

NC State University

Royal Holloway University of London

Korea University

Homepage

Preprint

How Can Software Engineering Help?

This paper's focus

Testing & Analysis — to uncover safety violations

Healing & Mitigation — to correct or reduce issues dynamically, without retraining

Falsification: Uncovering Safety Issues

Falsification: Uncovering Safety Issues

With many complex systems, full verification is often impractical

Falsification: Uncovering Safety Issues

With many complex systems, full verification is often impractical

Falsification flips the mindset:

"Can we find even one case where the system fails to meet its safety guarantees?"

Falsification: Uncovering Safety Issues

With many complex systems, full verification is often impractical

Falsification flips the mindset:

"Can we find even one case where the system fails to meet its safety guarantees?"

- Typical steps:
  a. Generate test cases (inputs or initial conditions)
  b. Run the AI controller
  c. Check if it violates a formal safety specification

Falsification: Uncovering Safety Issues

With many complex systems, full verification is often impractical

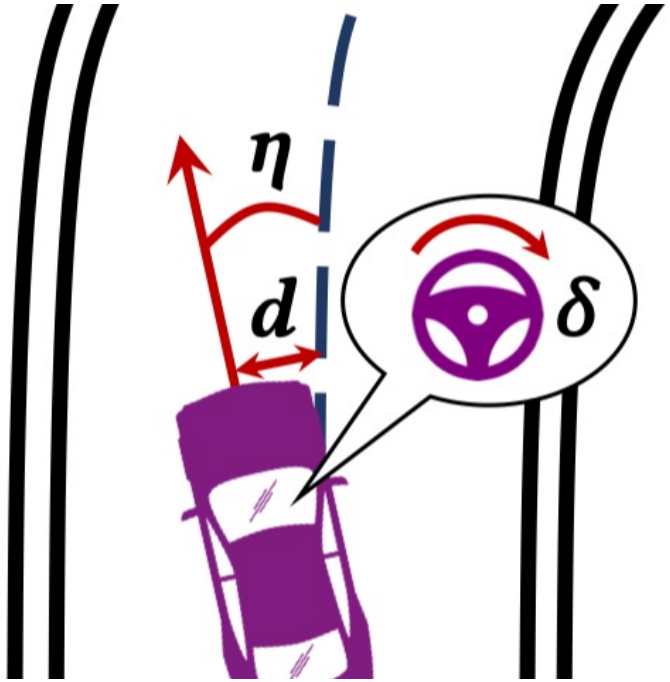
Falsification flips the mindset:

"Can we find even one case where the system fails to meet its safety guarantees?"

- Typical steps:
  a. Generate test cases (inputs or initial conditions)
  b. Run the AI controller
  c. Check if it violates a formal safety specification
- 🎯 If we find a violation, we've *falsified* the safety claim — and revealed a critical risk

What are We Falsifying for AI Controls?

- Self-driving cars:
  - Always stay within lane
  - Never exceed speed X
  - Avoid obstacles
  - …
- Robotic arms:
  - Avoid collisions with nearby workers or tools
  - Respect joint speed and torque limits
  - …



What Makes Falsification Harder for AI Controllers?

What Makes Falsification Harder for AI Controllers?

Scalability: AI controllers can be *4–50× slower* vs. classical ones

❌ Fewer test cases explored in fixed time → Missed violations

What Makes Falsification Harder for AI Controllers?

Scalability: AI controllers can be *4–50× slower* vs. classical ones

❌ Fewer test cases explored in fixed time → Missed violations

Diversity: When testing a complex safety spec (e.g., $\varphi_1 \wedge \varphi_2 \wedge \varphi_3$), naive search tends to *focus on the easiest subspec*

❌ Other specs remain untested

What Makes Falsification Harder for AI Controllers?

Scalability: AI controllers can be *4–50× slower* vs. classical ones

❌ Fewer test cases explored in fixed time → Missed violations

Diversity: When testing a complex safety spec (e.g., $\varphi_1 \wedge \varphi_2 \wedge \varphi_3$), naive search tends to *focus on the easiest subspec*

❌ Other specs remain untested

Falsifiers must be both *efficient* and *comprehensive*

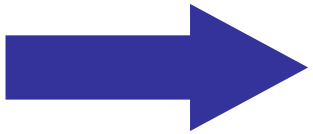
How Does Our Approach (Synthify) Address the Challenges?

Scalability

Diversity

How Does Our Approach (Synthify) Address the Challenges?

Scalability



Synthify constructs **proxy programs**

- Lightweight approximations of AI (RL) controllers
- Execute *more test cases* within the same time.

Diversity

How Does Our Approach (Synthify) Address the Challenges?

Scalability

Synthify constructs **proxy programs**

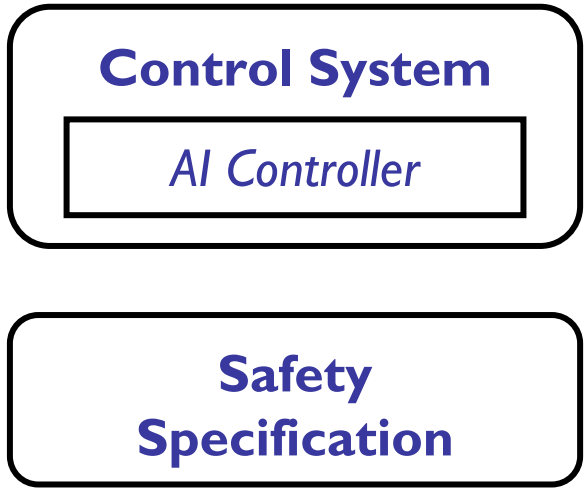
- Lightweight approximations of AI (RL) controllers
- Execute *more test cases* within the same time.

Diversity

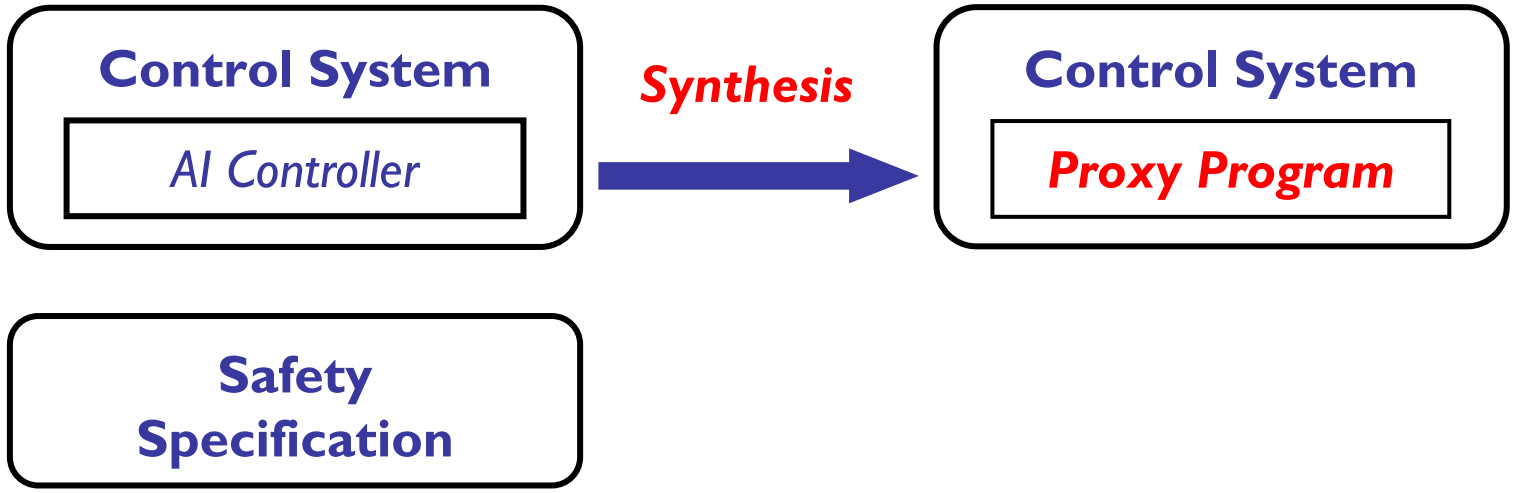
Synthify employs **subspec-switching**

- Breaks a spec to subspecs
- Dynamically changes subspec as falsification target
  - Employ random *explore-exploit* strategy
- Identify violations across subspecs

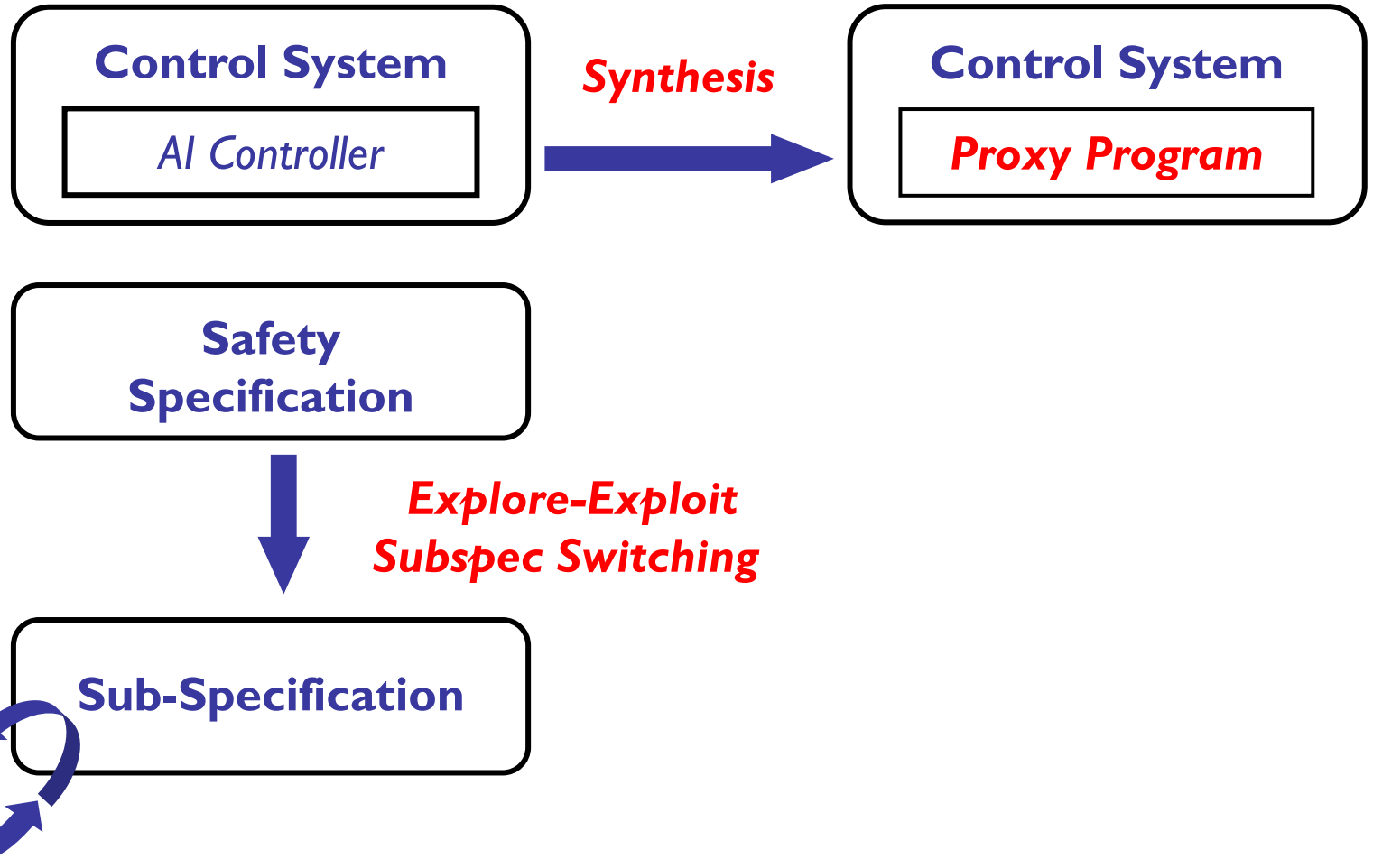
Synthify's Overall Workflow



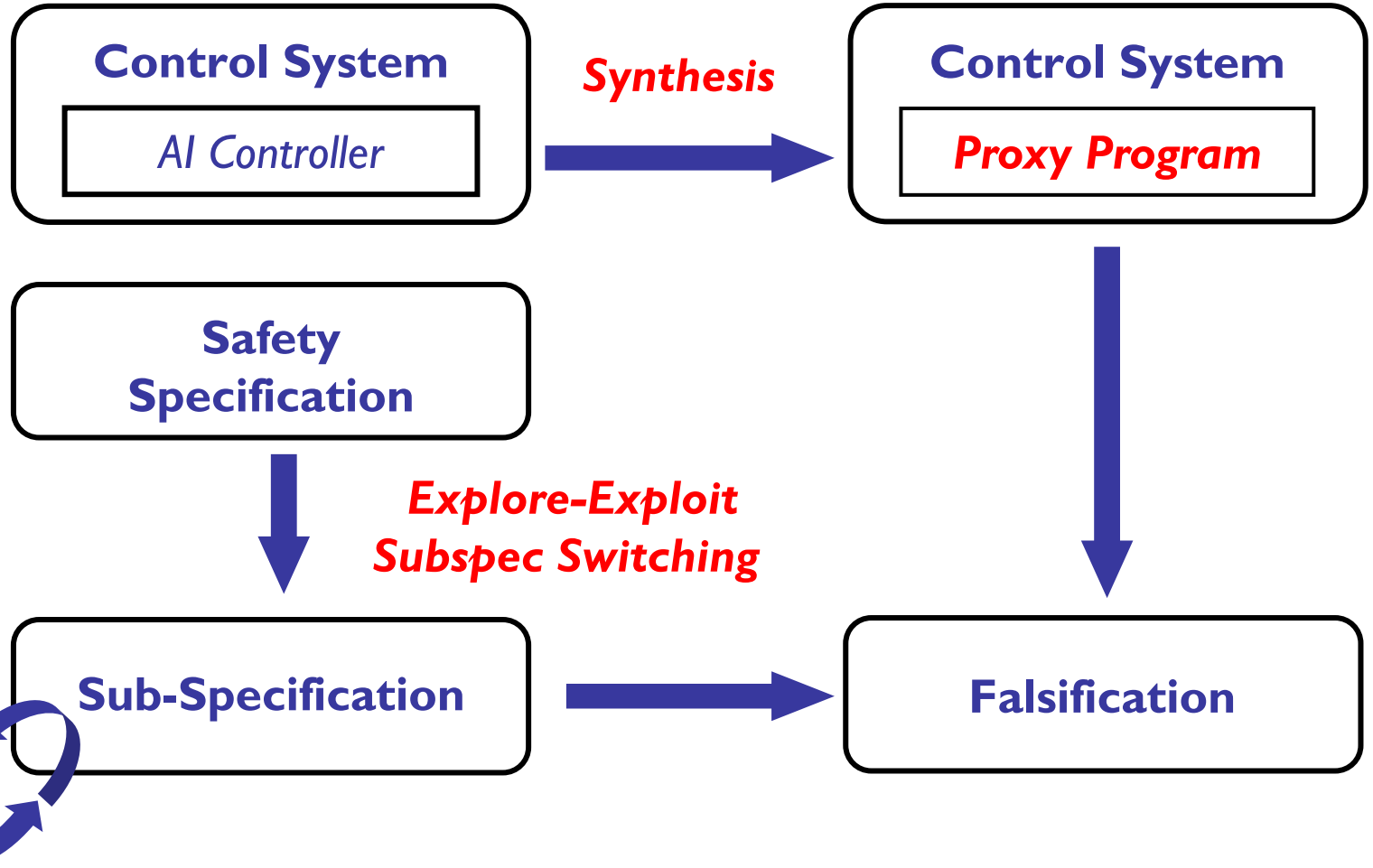
Synthify's Overall Workflow



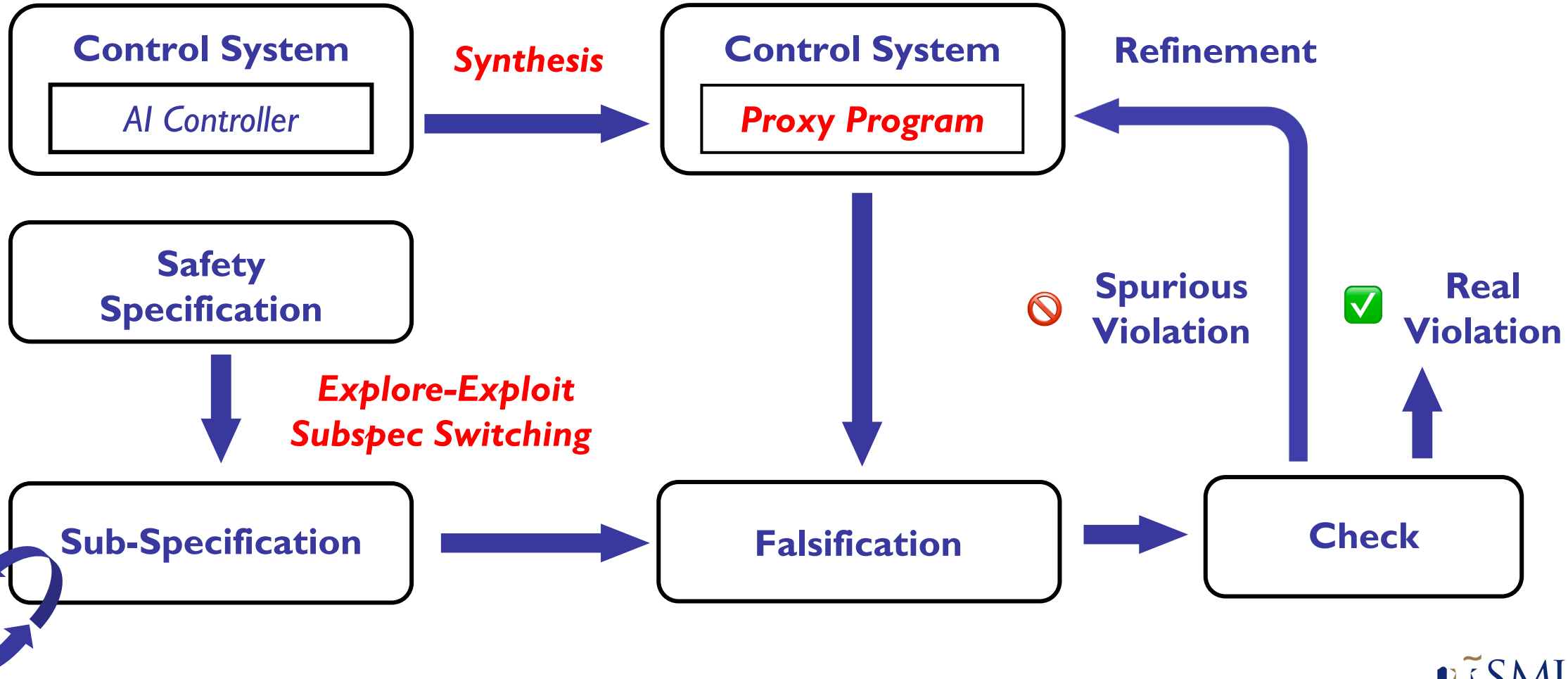
Synthify's Overall Workflow



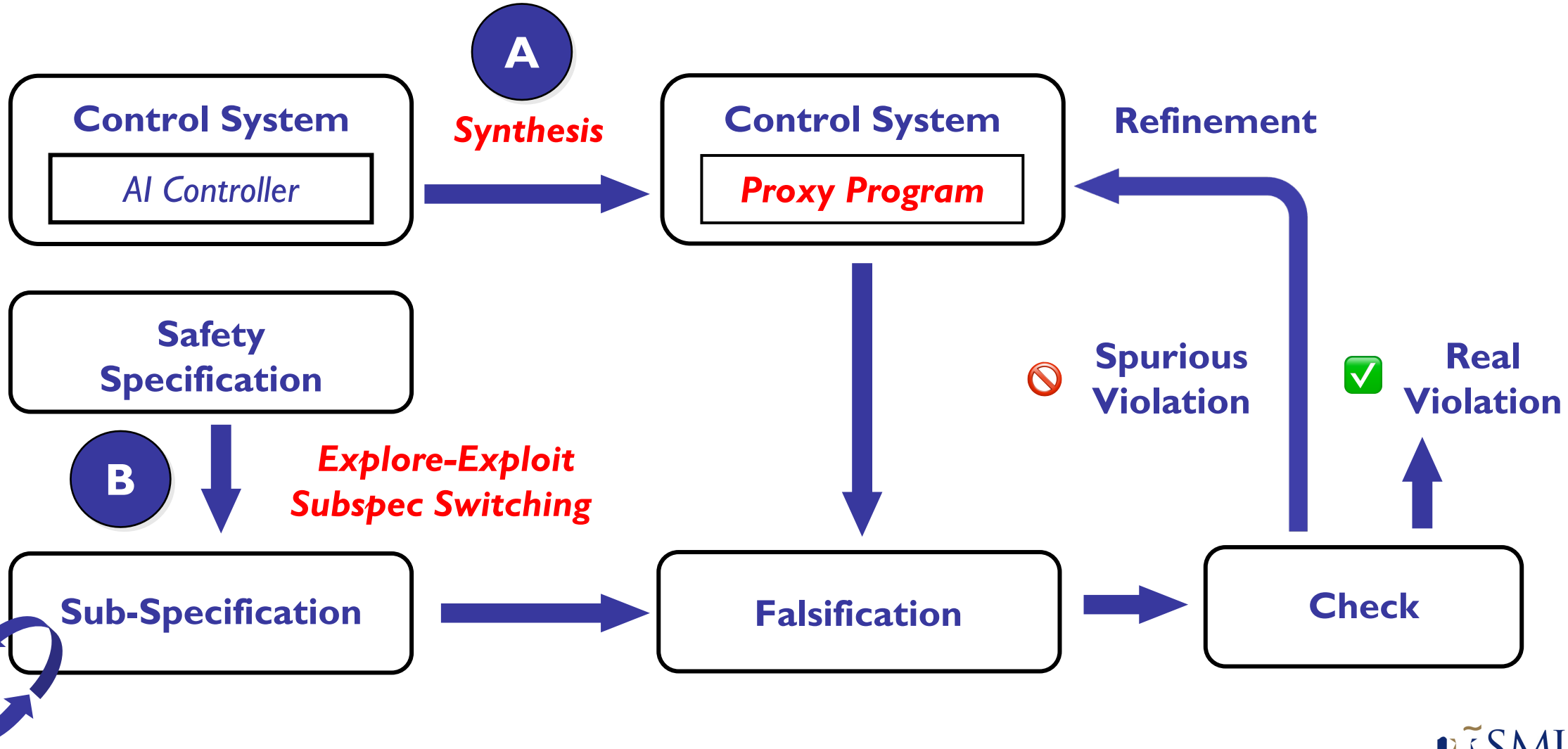
Synthify's Overall Workflow



Synthify's Overall Workflow



Synthify's Overall Workflow



A Synthesizing *Proxy Programs* for AI Controllers

```
1 def Proxy[θ](x_1, x_2, ..., x_n):
2     y_1 = θ_[1,1] * x_1 + θ_[1,2] * x_2 + ... + θ_[1,n] * x_n + θ_[1,n+1]
3     y_2 = θ_[2,1] * x_1 + θ_[2,2] * x_2 + ... + θ_[2,n] * x_n + θ_[2,n+1]
4     ...
5     y_m = θ_[m,1] * x_1 + θ_[m,2] * x_2 + ... + θ_[m,n] * x_n + θ_[m,n+1]
6     return y_1, y_2, ..., y_m
```

a *sketch* that implements a few linear controllers

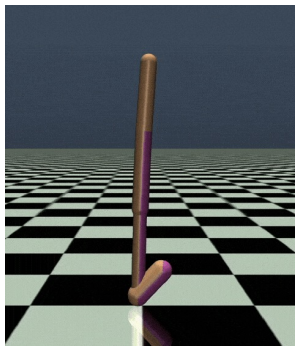
A Synthesizing *Proxy Programs* for AI Controllers

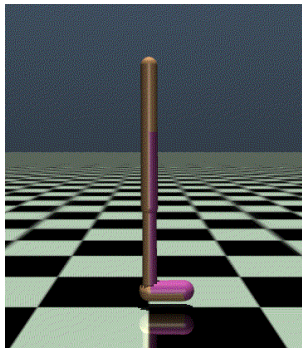
```
def Proxy[θ](x_1, x_2, ..., x_n):
    y_1 = θ_[1,1] * x_1 + θ_[1,2] * x_2 + ... + θ_[1,n] * x_n + θ_[1,n+1]
    y_2 = θ_[2,1] * x_1 + θ_[2,2] * x_2 + ... + θ_[2,n] * x_n + θ_[2,n+1]
    ...
    y_m = θ_[m,1] * x_1 + θ_[m,2] * x_2 + ... + θ_[m,n] * x_n + θ_[m,n+1]
    return y_1, y_2, ..., y_m
```

a *sketch* that implements a few linear controllers

- Much less computationally expensive to run
- Achieve similar functionality to some extent

AI can stand & walk





Linear controllers can also stand, acting similarly

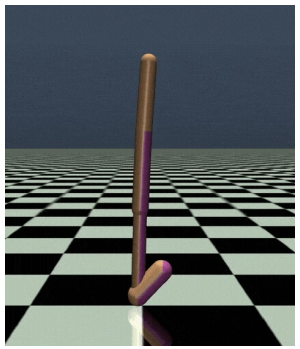
A Synthesizing *Proxy Programs* for AI Controllers

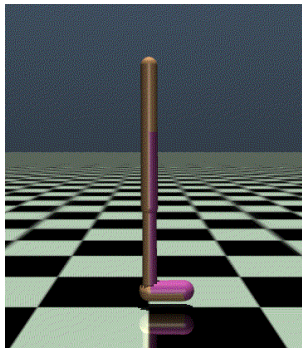
```
def Proxy[θ](x1, x2, ..., xn):
    y1 = θ[1,1] * x1 + θ[1,2] * x2 + ... + θ[1,n] * xn + θ[1,n+1]
    y2 = θ[2,1] * x1 + θ[2,2] * x2 + ... + θ[2,n] * xn + θ[2,n+1]
    ...
    ym = θ[m,1] * x1 + θ[m,2] * x2 + ... + θ[m,n] * xn + θ[m,n+1]
    return y1, y2, ..., ym
```

a *sketch* that implements a few linear controllers

- Much less computationally expensive to run
- Achieve similar functionality to some extent

AI can stand & walk





Linear controllers can also stand, acting similarly

Search-Based Synthesis

- ❑ randomly initializes θ
- ❑ obtains fitness $\boldsymbol{R}$ (distance between the outputs of the proxy program and AI controller), then updates θ

$$\theta \leftarrow \theta + \alpha \frac{1}{m\sigma} \sum_{i=1}^{m} \boldsymbol{N}_i \cdot \boldsymbol{R}_i$$

*m: population size, α: learning rate, σ: noise standard deviation, $\boldsymbol{N}$: random noise

- ❑ Repeat until max iters/convergence

B Explore-Exploit Subspec Switching

$$\varphi_{safety} \equiv \square_{[0,200]} \left(|\eta| < 90^\circ \wedge |d| < 1.0\right)$$

Complex specification

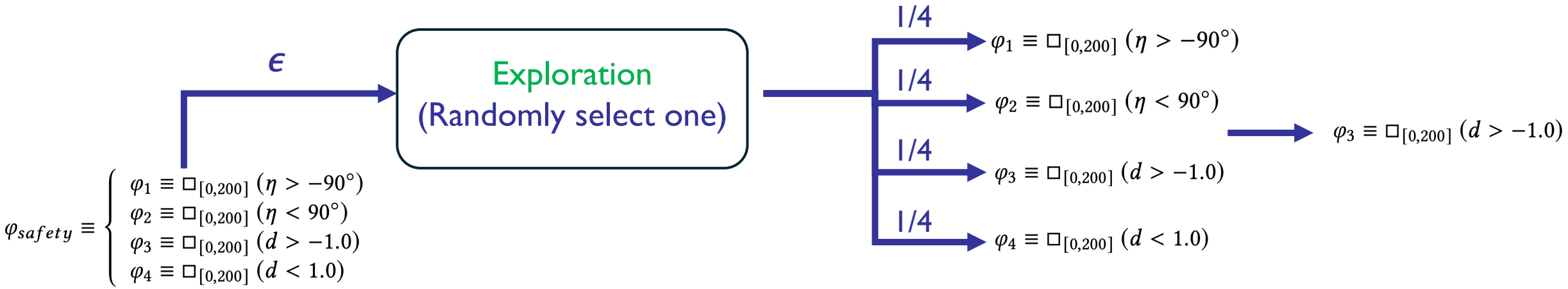
$$\varphi_{safety} \equiv \begin{cases} \varphi_1 \equiv \square_{[0,200]} \left(\eta > -90^\circ\right) \\ \varphi_2 \equiv \square_{[0,200]} \left(\eta < 90^\circ\right) \\ \varphi_3 \equiv \square_{[0,200]} \left(d > -1.0\right) \\ \varphi_4 \equiv \square_{[0,200]} \left(d < 1.0\right) \end{cases}$$

Multiple simpler sub-specifications

B Explore-Exploit Subspec Switching

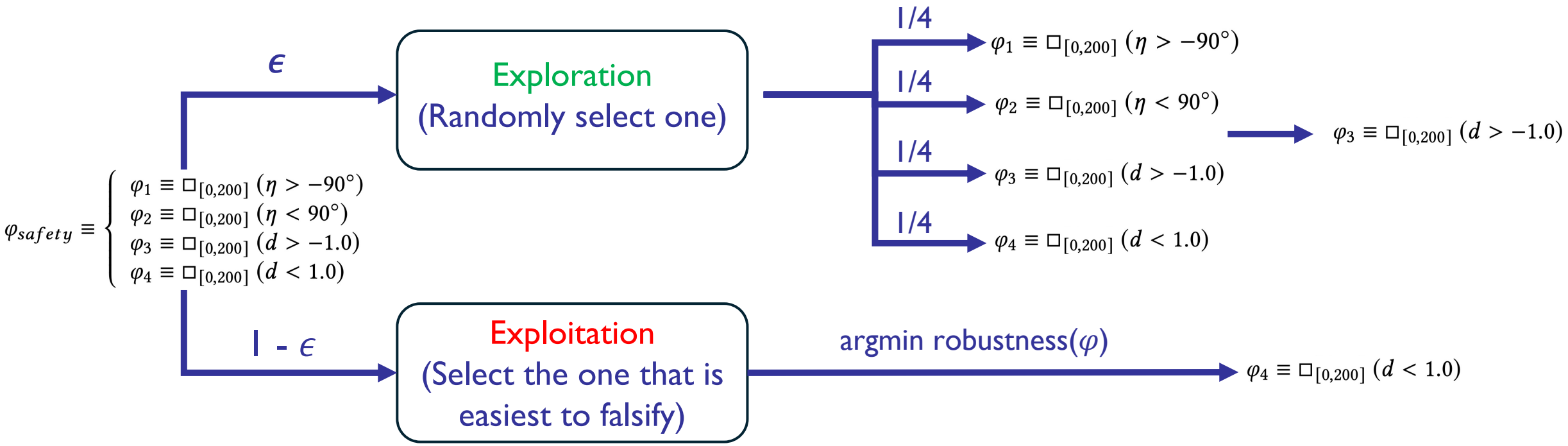
$$\varphi_{safety} \equiv \begin{cases} \varphi_1 \equiv \square_{[0,200]}\ (\eta > -90^\circ) \\ \varphi_2 \equiv \square_{[0,200]}\ (\eta < 90^\circ) \\ \varphi_3 \equiv \square_{[0,200]}\ (d > -1.0) \\ \varphi_4 \equiv \square_{[0,200]}\ (d < 1.0) \end{cases}$$

B Explore-Exploit Subspec Switching



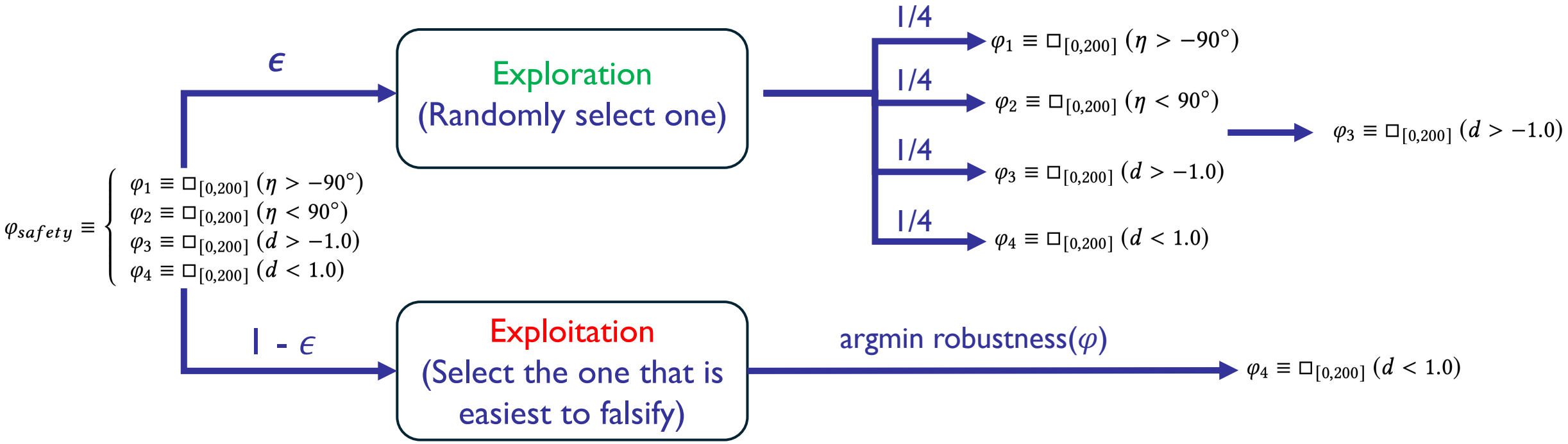
- With ϵ probability, randomly chooses a specification

B Explore-Exploit Subspec Switching



- With ϵ probability, randomly chooses a specification
- Or choose the one with the smallest robustness semantics (easiest to falsify based on historical trials)

B Explore-Exploit Subspec Switching



- With ϵ probability, randomly chooses a specification
- Or choose the one with the smallest robustness semantics (easiest to falsify based on historical trials)

Why exploit? One spec, many ways to fail — and we want to uncover them all

Results: Effectiveness & Efficiency

- Experiments on 8 AI control systems, e.g., Segway transporters, drones, and autonomous driving cars, etc.
- Comparison with a strong falsification tool: award-winning PSY-TaLiRo

Results: Effectiveness & Efficiency

- Experiments on 8 AI control systems, e.g., Segway transporters, drones, and autonomous driving cars, etc.
- Comparison with a strong falsification tool: award-winning PSY-TaLiRo

Efficacy
Synthify reveals **7.8× more** safety violations than PSY-TaLiRo

Results: Effectiveness & Efficiency

- Experiments on 8 AI control systems, e.g., Segway transporters, drones, and autonomous driving cars, etc.
- Comparison with a strong falsification tool: award-winning PSY-TaLiRo

Efficacy
Synthify reveals **7.8× more** safety violations than PSY-TaLiRo

Efficiency
Synthify can find a violation **12.8× faster** than PSY-TaLiRo

Results: Effectiveness & Efficiency

- Experiments on 8 AI control systems, e.g., Segway transporters, drones, and autonomous driving cars, etc.
- Comparison with a strong falsification tool: award-winning PSY-TaLiRo

Efficacy

Synthify reveals **7.8× more** safety violations than PSY-TaLiRo

Subspecification Coverage

Synthify falsify **137.7% more** sub-specifications than PSY-TaLiRo

Efficiency

Synthify can find a violation **12.8× faster** than PSY-TaLiRo

Results: Effectiveness & Efficiency

- Experiments on 8 AI control systems, e.g., Segway transporters, drones, and autonomous driving cars, etc.
- Comparison with a strong falsification tool: award-winning PSY-TaLiRo

Efficacy
Synthify reveals **7.8× more** safety violations than PSY-TaLiRo

Efficiency
Synthify can find a violation **12.8× faster** than PSY-TaLiRo

Subspecification Coverage
Synthify falsify **137.7% more** sub-specifications than PSY-TaLiRo

Proxy Program Quality
Only **2.1 (of 50) spurious violations** occurs, proxy programs reflects AI's safety properties at **95.8%**

Synthesizing Efficient and Permissive **Programmatic Runtime Shields** for Neural Policies

ACM Transactions on Software Engineering and Methodology (TOSEM), 2025


Jieke Shi


Junda He


Zhou Yang


Đorđe Žikelić


David Lo

SMU SINGAPORE MANAGEMENT UNIVERSITY

UNIVERSITY OF ALBERTA

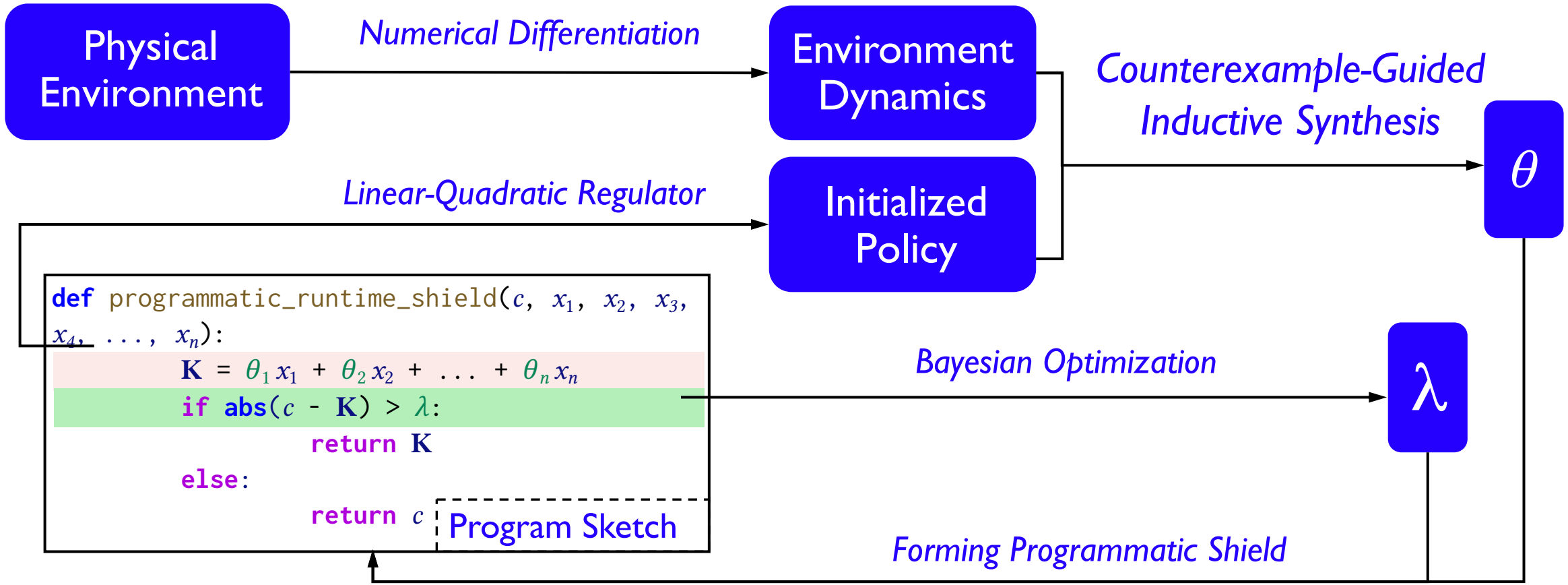
Homepage

Preprint

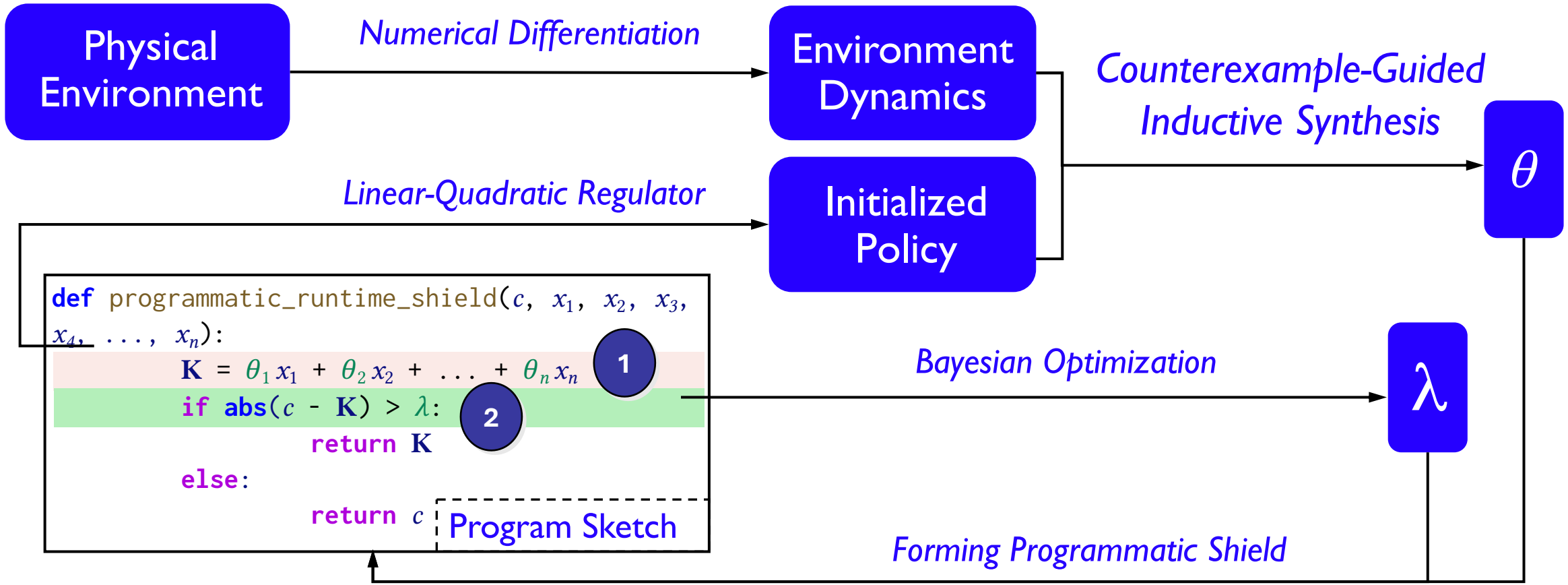
How to Get Guaranteed Safe Commands?

- Approximate complex AI models with a simple linear policy (simple program)
- Simple linear policy can be input to verification tools
  - However, may not produce optimal commands
- Dynamically switch between AI output and linear policy output
  - Minimize switching to only when interventions are needed

Proposed Approach: Aegis



Proposed Approach: Aegis



1 Synthesizing A Safe Linear Policy

```
1  def programmatic_runtime_shield(c, x_1, x_2, ..., x_n):
2      K = θ_1 x_1 + θ_2 x_2 + ... + θ_n x_n
3      if abs(c − K) > λ:
4          return K
5      else:
6          return c
```

Synthesize coefficients of the linear policy and guarantee it can output safe commands

Step 1A: Inferring Environment (State-Transition) Dynamics

- The environment's behavior is often complex and unknown (black box)
- Intuition:
  - We poke the environment with small perturbations and observe how it reacts
  - From that, we build a simple linear approximation of its behavior

1 Synthesizing A Safe Linear Policy

```
1  def programmatic_runtime_shield(c, x_1, x_2, ..., x_n):
2      K = θ_1 x_1 + θ_2 x_2 + ... + θ_n x_n
3      if abs(c − K) > λ:
4          return K
5      else:
6          return c
```

Synthesize coefficients of the linear policy and guarantee it can output safe commands

Step 1B: Counterexample-guided Inductive Synthesis

- Propose candidate coefficients
- Simulate the shielded system using the approximated environment model (**1A**)
- Collect counterexamples where safety is violated
- Refine the shield to eliminate those violations

2 Synthesizing A Switching Strategy

```
1  def programmatic_runtime_shield(c, x_1, x_2, ..., x_n):
2      K = θ_1 x_1 + θ_2 x_2 + ... + θ_n x_n
3      if abs(c − K) > λ:
4          return K
5      else:
6          return c
```

Synthesize switching strategy in a form of linear inequality with optimized threshold

- λ is learned to minimize unnecessary interventions

$$\arg\min_{\lambda} \left(\log(\mathcal{V} + 1) - \frac{\mathcal{V}^*}{\mathcal{I} + 1} \right)$$

objective function of our *Bayesian optimization*

- V: the number of violations
- V∗: the number of necessary interventions
- I: the total number of interventions

Limitation of SOTA & Benefit of Aegis

```
1  def programmatic_runtime_shield(c, η, ω):
2          η, ω = environment_model(c, η, ω)
3          if 8.35η^4 - 0.01η^3ω + 0.05η^2ω^2 - 0.09ηω^3 +
                   1.25ω^4 + 2.24η^3 + 0.07η^2ω - 0.02ηω^2 +
                   0.22ω^3 + 5.05η^2 - 3.08ηω + 4.65ω^2 -
                   1.89η + 6.75ω - 16.13 > 0:
4                  return -0.12η - 0.97ω
5          else:
6                  return c
```

Synthesized by VRL (PLDI'19, ACM SIGPLAN Distinguished Paper)

- High-degree polynomials -- large time & memory overhead
- Cause many unnecessary interventions (block safe commands)

Limitation of SOTA & Benefit of Aegis

```
1 def programmatic_runtime_shield(c, η, ω):
2         η, ω = environment_model(c, η, ω)
3         if 8.35η^4 - 0.01η^3ω + 0.05η^2ω^2 - 0.09ηω^3 +
                  1.25ω^4 + 2.24η^3 + 0.07η^2ω - 0.02ηω^2 +
                  0.22ω^3 + 5.05η^2 - 3.08ηω + 4.65ω^2 -
                  1.89η + 6.75ω - 16.13 > 0:
4                 return -0.12η - 0.97ω
5         else:
6                 return c
```

Synthesized by VRL (PLDI'19, ACM SIGPLAN Distinguished Paper)

- High-degree polynomials -- large time & memory overhead
- Cause many unnecessary interventions (block safe commands)

```
1 def programmatic_runtime_shield(c, η, ω):
2         κ = -0.48η - 1.29ω
3         if abs(c - κ) > 0.59:
4                 return κ
5         else:
6                 return c
```

Synthesized by Aegis for the same neural policy

Linear inequality -- lightweight & optimized for minimizing unnecessary interventions

Results: Effectiveness, Overhead, Permissiveness, Scalability

- Experiments on AI control systems, e.g., Segway transporters, drones, and self-driving cars, etc.
- Comparison with SOTA: VRL (PLDI'19, ACM SIGPLAN Distinguished Paper)

Results: Effectiveness, Overhead, Permissiveness, Scalability

- Experiments on AI control systems, e.g., Segway transporters, drones, and self-driving cars, etc.
- Comparison with SOTA: VRL (PLDI'19, ACM SIGPLAN Distinguished Paper)

Safety Mitigation
Aegis corrects all (**100%**) unsafe commands, letting systems operate without any violations

Results: Effectiveness, Overhead, Permissiveness, Scalability

- Experiments on AI control systems, e.g., Segway transporters, drones, and self-driving cars, etc.
- Comparison with SOTA:VRL (PLDI'19, ACM SIGPLAN Distinguished Paper)

Safety Mitigation
Aegis corrects all (**100%**) unsafe commands, letting systems operate without any violations

Overhead
Aegis incurs **5.0% time** and **0.61MB memory overhead**, a **2.1×** and **4.4×** improvement over VRL

Results: Effectiveness, Overhead, Permissiveness, Scalability

- Experiments on AI control systems, e.g., Segway transporters, drones, and self-driving cars, etc.
- Comparison with SOTA:VRL (PLDI'19, ACM SIGPLAN Distinguished Paper)

Safety Mitigation
Aegis corrects all (**100%**) unsafe commands, letting systems operate without any violations

Overhead
Aegis incurs **5.0% time** and **0.61MB memory overhead**, a **2.1×** and **4.4×** improvement over VRL

False Alarms
Aegis incurs **1.6× less** unnecessary interventions over VRL

Results: Effectiveness, Overhead, Permissiveness, Scalability

- Experiments on AI control systems, e.g., Segway transporters, drones, and self-driving cars, etc.
- Comparison with SOTA:VRL (PLDI'19, ACM SIGPLAN Distinguished Paper)

Safety Mitigation

Aegis corrects all (**100%**) unsafe commands, letting systems operate without any violations

Overhead

Aegis incurs **5.0% time** and **0.61MB memory overhead**, a **2.1×** and **4.4×** improvement over VRL

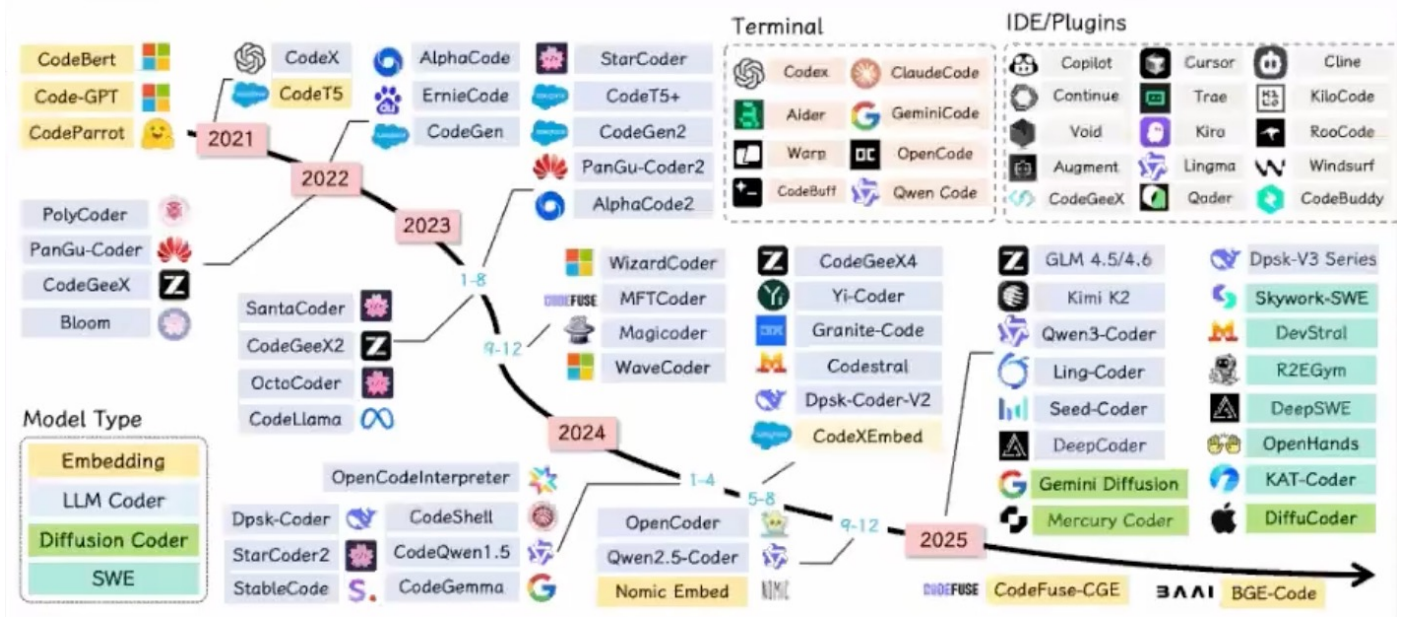
False Alarms

Aegis incurs **1.6× less** unnecessary interventions over VRL

Scalability

Aegis requires **65s** for synthesizing a shield, which **reduces 77.3%** of the time cost

Future Work: Is Software Engineering Dying?



Staying true to the original while embracing change

Software Engineering is always on the way to 'killing' itself.

Birth → With development in automation → Death

What is the unchanged?

Programming Languages/AI needs compilers, but compilers are complicated

- It can crash
- It can produce wrong results/slow code
- It can cause vulnerabilities

What is the unchanged?

Mut4All: Fuzzing Compilers via LLM-Synthesized Mutators Learned from Bug Reports

BO WANG, Beijing Jiaotong University, China
PENGYANG WANG, Beijing Jiaotong University, China
CHONG CHEN, Beijing Jiaotong University, China
MING DENG, Beijing Jiaotong University, China
JIEKE SHI, Singapore Management University, Singapore
QI SUN, Beijing Jiaotong University, China
CHENGRAN YANG, Singapore Management University, Singapore
ZHOU YANG, University of Alberta, Canada
YOUFANG LIN, Beijing Jiaotong University, China
JUNJIE CHEN, Tianjin University, China
JUN SUN, Singapore Management University, Singapore
DAVID LO, Singapore Management University, Singapore

6 Feb 2026

Mutation-based fuzzing has proven effective in uncovering compiler bugs. However, designing high-quality

TOSEM: Agentic AI Special Issue

Find 158 bugs in gcc/rustc

Optimizing and Fortifying AI-Assisted Software Ad Absurdum

Jieke Shi
https://jiekeshi.notion.site
PhD Candidate & Research Engineer
Software Analytics Research Group (SOAR)
School of Computing and Information Systems
Singapore Management University

Introduction

Recent years have witnessed a surge in the integration of Artificial Intelligence (AI) into software development pipelines, giving rise to a new class of software that is either generated or co-authored by AI models such as Large Language Models (LLMs), or incorporates embedded calls to LLMs and neural policies at runtime. We refer to this emerging class as *AI-assisted software—programs that are either produced by AI or tightly coupled with AI models as integral components*. This trend is particularly evident

NRF-PA Proposal

LLMs can benefit from compiler tuning

School of Computing and Information Systems

SMU SINGAPORE MANAGEMENT UNIVERSITY

What is the unchanged?

Programming Languages/AI needs compilers, but compilers are complicated

- It can crash
- It can produce wrong results/slow code
- It can cause vulnerabilities

Co-evolving compilers and AI so they optimize each other

What is the changed?

What is the changed?

The way to develop software has changed, i.e., using AI, but can we trust AI to process code?

What is the changed?

Optimizing and Fortifying AI-Assisted Software Ad Absurdum

Jieke Shi
https://jiekeshi.notion.site
PhD Candidate & Research Engineer
Software Analytics Research Group (SOAR)
School of Computing and Information Systems
Singapore Management University

Introduction

Recent years have witnessed a surge in the integration of Artificial Intelligence (AI) into software development pipelines, giving rise to a new class of software that is either generated or co-authored by AI models such as Large Language Models (LLMs), or incorporates embedded calls to LLMs and neural policies at runtime. We refer to this emerging class as *AI-assisted software—programs that are either produced by AI or tightly coupled with AI models as integral components*. This trend is particularly evident

NRF-PA Proposal

Towards more trustworthy, and synergistic AI-assisted software

- Build more powerful foundation models for understanding code/user intentions at scale
- Build more reliable and safety-enforcing AI tools
- Society-related analysis: OSS, education,…

Dissertation Defense

Optimizing & *Fortifying* AI Software through the Lens of Artifact Synthesis

Jieke Shi
Final-year PhD Candidate at SMU
AI & Software

jiekeshi@smu.edu.sg
https://jiekeshi.tech

SMU
SINGAPORE MANAGEMENT UNIVERSITY

Committee Members:

David LO (Supervisor/Chair)
Lingxiao JIANG
Xiaofei XIE
Zhenchang XING